EÖTVÖS LORÁND UNIVERSITY

FACULTY OF INFORMATICS

DEPARTMENT OF ARTIFICIAL INTELLIGENCE

Smart Parking License Plate Recognition & Dynamic Pricing System

Supervisor: ***Guettala Walid***
PhD Student

Author: ***Md Farid***
B.Sc Computer Science

Budapest, 2026

EÖTVÖS LORÁND UNIVERSITY
FACULTY OF INFORMATICS

Thesis Topic Registration Form

Student's Data:
Student's Name: Md Farid
Student's Neptun code: WEFKHB

Educational Information:
Training programme: Computer Science BSc

I have an internal supervisor

Internal Supervisor's Name: *Guettala walid*
Supervisor's Home Institution: ***Department of Artificial Intelligence***
Address of Supervisor's Home Institution: ***1117, Budapest, Pázmány Péter sétány 1/C.***
Supervisor's Position and Degree: *PhD Candidate*

Thesis Title: Smart Parking License Plate Recognition & Dynamic Pricing System

Topic of the Thesis:
(Upon consulting with your supervisor, give a 150-300-word-long synopsis os your planned thesis.)

Problem to be solved:
Urban parking management relies heavily on manual processes and fixed pricing, making enforcement inefficient and prone to errors. Current systems often lack automation for vehicle identification, dynamic fee calculation, and administrative control, leading to poor user experience and operational inefficiencies.
Motivation:
Automating license plate recognition and implementing dynamic pricing can improve efficiency, fairness, and revenue optimization. Using pre-trained models allows rapid prototyping of a complete system that combines computer vision, business logic, and a web interface, giving practical experience in integrating AI components without complex model training.
Method of operation:
The system will use pre-trained YOLO for license plate detection and EasyOCR for text recognition. A synthetic dataset will simulate parking zones, sessions, and vehicle registries. The fee engine calculates dynamic rates based on zone, time, and repeat offenses, while all data is persisted in a relational database.
Workflow:
Image upload → plate detection → OCR → plate validation → lookup plate history → dynamic fee calculation → store session → display results via web UI → admin
monitoring and reporting.
Implementation:
A web-based application with a frontend (React/Streamlit) and a Python backend will demonstrate the pipeline. The system includes plate recognition, fee calculation, database operations, and an admin dashboard. Evaluation will measure detection accuracy, fee calculation correctness, response time, and usability.
Output:
The project will deliver a functional web application, synthetic dataset and generation scripts, technical documentation, evaluation results, and a thesis report summarizing implementation, findings, and recommendations for future real-world deployment.

Budapest, 2025. 10. 11.

Contents

Chapter 1

1. Introduction

1.1 Motivation

In growing urban areas, parking management remains a daily challenge for both drivers and parking authorities. Drivers often spend unnecessary time searching for available parking spaces, which increases road congestion, causes frustration, and wastes fuel. At the same time, parking workers and administrators must handle vehicle records, payments, fines, and zone monitoring in a way that is often slow when supported only by manual processes. These difficulties show the need for a smarter and more organized parking solution that can reduce confusion and improve the overall flow of parking-related operations.

The motivation for this project comes from the need to make parking management more efficient, transparent, and practical through a unified digital system. In addition to handling parking sessions, vehicles, payments, and administrative tasks, the project is motivated by the idea of giving users better situational awareness through live map support and congestion-related information. A system that helps visualize parking zones and activity in real time can support better decisions for drivers and can also help administrators monitor high-demand areas more effectively. This makes the parking process easier not only for individual users, but also for the people responsible for supervising and managing the service.

Modern web and software technologies make it possible to build such a system in a practical and accessible way. By using FastAPI for backend services, React for the frontend interface, SQLite for structured local data storage, and computer vision tools such as YOLO and EasyOCR for license plate support, the project combines everyday usability with intelligent automation. The motivation behind choosing these technologies is not only technical implementation, but also the need to create a responsive, web-based platform that can support real-time interaction, simplify parking-related workflows, and contribute to a more efficient urban parking experience.

1.2 Goals

The primary objective of this project is to design and implement a practical Smart Parking System that can support everyday parking operations in budapest through one connected web application. The work combines parking sessions, user roles, supervision, payments, worker activity, and reporting inside one platform. In this way, the system is intended to reduce manual administration, improve operational clarity, and provide a software foundation that can be reviewed, maintained, and extended in a structured manner.

1. **Efficient Parking Operations:** Parking-related tasks should move through a clearer digital workflow. The system should reduce delays, confusion, and unnecessary manual coordination in daily use.

2. **Structured Session Control:** Vehicle and parking session data should remain consistent and easy to track. Starting, monitoring, and closing sessions should therefore follow one transparent process.

3. **Secure Role-Based Access:** Drivers, workers, and administrators need different system functions. Access should be controlled clearly so that each role works only with the parts relevant to its responsibilities.

4. **Administrative Supervision:** The platform should support monitoring at management level as well as everyday parking use. Records, summaries, and operational views should help administrators supervise the system more effectively.

5. **Payment and Notice Management:** Payment-related records and notice handling should be integrated into the same platform. This keeps financial and enforcement-related information easier to manage and review.

6. **Live Monitoring Support:** Real-time understanding of parking activity should be improved through map-based visibility and congestion-related information. This gives the system a more practical role than a simple record-keeping application.

7. **Scalable and Reliable Architecture:** The software should remain stable in current use while still supporting later extension. Scalability and reliability should be supported by FastAPI, React, SQLite, together with YOLO and EasyOCR for intelligent plate-related functionality.

8. **Comprehensive Testing and Quality Assurance:** Reliable system behavior should be checked through backend tests, frontend validation, and API-level verification of important workflows. Treating testing as a project goal strengthens the technical quality of the final result.

9. **Detailed and Accessible Documentation:** The project should include clear documentation for both users and developers. Installation, configuration, usage, and maintenance-related information should be presented in a structured form.

By accomplishing these goals, the project is intended to provide a practical and technically structured parking management solution that is useful in its present form and suitable for later extension.

1.3 Scope of the Project

The Smart Parking System provides a practical and comprehensive parking management solution for drivers, parking workers, and administrators in budapest. District-oriented parking operation, role-based workflows, administrative supervision, payment-related functions, messaging, reporting, and selected intelligent automation are all part of the system. The scope encompasses the following core functionalities and components:

1. **Budapest District-Based Operation:** The system is tailored to budapest and its

district-based parking environment. Districts and zone-related records are treated as core parts of the software structure.

2. **Role-Based Workflows:** Separate workflows are provided for the three main roles supported by the application.
   - **User:** manages personal vehicles, parking sessions, payments, notices, and message-related updates.
   - **Admin:** maintains districts, supervises records, reviews reports, manages workers and users, and can directly close sessions.
   - **Worker:** patrols the city, scans vehicles, verifies session status, and can issue fines when no active session exists.
3. **Vehicle and Session Management Modules:** Vehicle registration, session creation, running session tracking, and session closure are included in the system. These modules form the main operational foundation of the parking workflow.
4. **Payment and Notice Records:** Payment-related information and notice management are both covered within the scope. The system therefore handles not only standard parking use but also enforcement-related record keeping.
5. **Administrative Views and Reporting:** Administrative pages are available for supervision, summaries, and report-oriented review. These functions cover parking activity, notices, user and worker records, and revenue-related information.
6. **Map-Based Visualization:** A live map is included so that parking-related information can be viewed visually rather than only through text-based records. This supports clearer operational awareness during use.
7. **Congestion-Related Visibility:** Congestion-oriented presentation of activity is also included. This helps higher-demand parking areas stand out more clearly.
8. **Message and Notification Support:** A dedicated message section is available for notification-style communication and for delivering important updates to the relevant users.
9. **Plate Scanning and OCR Support:** Worker-oriented scanning functions are available together with upload-based plate processing.
   - YOLO is used to support plate detection within uploaded vehicle images.
   - EasyOCR is used to read the detected plate text during verification tasks.
10. **Authentication and Access Control:** Login, registration, and role-based access handling are included as core security features. User management and protected access to role-specific pages are both part of the system.

11. **Testing Coverage:** The scope also includes the testing work completed during development.
    - Backend logic is checked through automated tests.
    - Frontend behavior is supported by validation-oriented testing.
    - Important API workflows are verified at endpoint level.
12. **Documentation Coverage:** Both user-facing and developer-facing documentation are included. User documentation covers setup and everyday usage, while developer documentation describes system structure, workflow, testing, and technical details needed for maintenance or later extension.

Chapter 2

2. User Documentation

2.1 Introduction

Welcome to the Smart Parking Project. This web application is designed to support the management of parking activity in Budapest districts through a single integrated platform. The system combines user-facing parking services with administrative supervision and worker patrol functions, making it possible to handle different parking-related tasks within one environment. Drivers can register vehicles, start parking sessions, monitor running costs, and settle unpaid fees. Administrators can maintain districts, review traffic and revenue statistics, and supervise the overall operation of the platform. Parking workers can check plate numbers, verify whether a vehicle has an active session, and issue a fine when necessary.

The main user-facing idea behind the system is simplicity. Instead of relying on separate tools for parking, payment handling, district lookup, and notifications, the platform brings these functions together in one application. The interface adapts to the role of the logged-in user, while the underlying data remain consistent across the whole system. This allows the application to provide a more organized, practical, and efficient parking management experience for all types of users.

2.2 Used Methods and Tools

The system integrates a combination of practical frontend, backend, database, image-processing, and development tools to support parking sessions, district-based operation, payments, reporting, worker patrol tasks, and image-based plate verification. Below is a structured summary of the main tools and methods used in the project:

1. **Frontend Technologies:**
    1. **React.js** – Provides an interactive and component-based frontend experience.
    2. **Vite** – Supports fast frontend development and efficient project bundling.
    3. **React Router** – Handles navigation between dashboards, session pages, payment pages, reports, worker tools, and profile pages.
    4. **Chart.js and react-chartjs-2** – Used to visualize parking and reporting information through charts.
    5. **React Leaflet and Leaflet** – Provide map-based visualization of districts, zones, and congestion-related information.
2. **Backend Technologies:**
    1. **FastAPI** – Provides fast, reliable, and structured backend API functionality.

2. **Pydantic** – Ensures request and response validation through defined schemas.
3. **Python** – Used for implementing backend services, business logic, repositories, and utility modules.
4. **SQLite** – Stores users, vehicles, sessions, zones, payments, notices, and reporting data in a structured relational database.

3. **Image Processing and OCR Tools:**
   1. **OpenCV** – Used for image decoding and image-handling tasks related to uploaded plate images.
   2. **EasyOCR** – Extracts text from uploaded vehicle plate images.
   3. **Ultralytics YOLOv8** – Used for license plate localization. A YOLOv8 model fine-tuned for license plates returns bounding boxes for plate regions, which are then passed to EasyOCR.

4. **Development and Testing Tools:**
   1. **Git and GitHub** – Support version control and project collaboration.
   2. **Visual Studio Code** – Used as the main code editor and development environment.
   3. **Pytest** – Used for backend unit and service-level testing.
   4. **Vitest and Testing Library** – Used for selected frontend test cases and component validation.
   5. **Postman** – Used for manual API endpoint testing and request/response validation.

2.3 User Guide

2.3.1 Hardware Requirements

The Smart Parking web application can run on systems with the following hardware configurations:

Component	Minimum	Recommended
Processor	2.0 GHz dual-core CPU	3.0 GHz or faster quad-core CPU
Memory	4 GB RAM	8 GB RAM or higher
Display	1024×768 resolution	1920×1080 resolution (Full HD)
Storage	2 GB available disk space	SSD with additional free space

Table 1: Hardware requirements

Running the application on systems below these minimum requirements may lead to slower frontend loading, delayed API responses, or reduced performance when optional OCR-related functions are used.

2.3.2 Network Requirements

For normal usage, the network connection should satisfy the following conditions:

- Bandwidth: 50 KBps (400 kbps) or higher
- Latency: Under 150 ms
- Stable local or internet access when frontend and backend are not running on the same machine

2.3.3 Supported Web Browsers

The platform is compatible with the following browsers:

- Microsoft Edge (latest version)
- Mozilla Firefox (latest version)
- Google Chrome (latest version)

2.3.4 Roles

The application supports three roles.

Table 2: Roles and their main tasks

Role	Main tasks
User	manages own vehicles, opens sessions, checks notices, reviews history, and pays fees
Admin	monitors activity, manages districts, reviews financial summaries, and sends messages
Worker	checks parked vehicles, verifies session status, and issues fines when needed

2.3.5 Basic Navigation

After login, the visible pages depend on the role. A user normally works with the dashboard, vehicles, sessions, payment, profile, and messages pages. An administrator sees management-oriented pages such as zones, reports, workers, and user summaries. A worker sees the patrol workflow and the message-related pages required for field work.

2.4 Installation and Configuration

Prerequisites

Before installation, ensure your system includes:

- **Git:** For version control and repository management.

- **Python (3.9 or higher):** Required for running backend services.
- **Node.js and npm:** Required for running the frontend application.

Installing Git

1. Download Git from https://git-scm.com/downloads
2. Follow the on-screen instructions for your operating system.

Installing Python

1. Visit https://www.python.org/downloads/ to install Python (3.9 or later).
2. Verify installation via command prompt:

```
python --version
```

Setup Instructions

Cloning the Repository

1. Open terminal or command prompt.
2. Navigate to your preferred directory:

```
cd ~/your-directory
```

3. Clone repository:

```
git clone https://szofttech.inf.elte.hu/gnn/thesis/farid-md-farid.git
cd farid-md-farid
```

Setting Up the Environment

1. Create a virtual environment:

```
python -m venv venv
```

2. Activate virtual environment:

Windows:

```
venv\Scripts\activate
```

Mac/Linux:

```
source venv/bin/activate
```

3. Install dependencies:

```
cd backend
pip install -r requirements-ci.txt
# optional for OCR / plate detection support:
pip install opencv-python-headless easyocr ultralytics numpy
cd ..
```

Running the Application

1. Ensure virtual environment activation.
2. Navigate to project folder.
3. Run backend server (FastAPI):

```
cd backend
uvicorn app.main:app --reload
```

4. Run frontend application:

Open a new terminal and in the frontend directory run the following commands:

```
cd frontend
npm install
# if version conflicts occur:
# npm install --legacy-peer-deps
npm install
npm run dev
```

If local configuration variables are needed, create the appropriate environment file before starting the frontend. Store only the values required by your own local setup, and keep sensitive credentials outside version control.

2.5 Features of the Smart Parking Project

The Smart Parking Project includes several practical features for users, administrators, and parking workers. The following sections describe the most important parts of the system in the same user-oriented style as the sample documentation.

2.5.1 User Authentication and Login

What It Does: This feature allows users to create an account, sign in, and access the system according to their assigned role. After login, the interface changes depending on whether the account belongs to a driver, an administrator, or a parking worker. Special access codes are required during registration to create administrator or worker accounts, ensuring that these roles are assigned securely.

How to Use It:

1. Open the login or registration page.
2. Enter the required account details.
3. If registering as an administrator or worker, provide the special access code.
4. Submit the form to sign in or create a new account.
5. Continue to the dashboard after successful authentication.

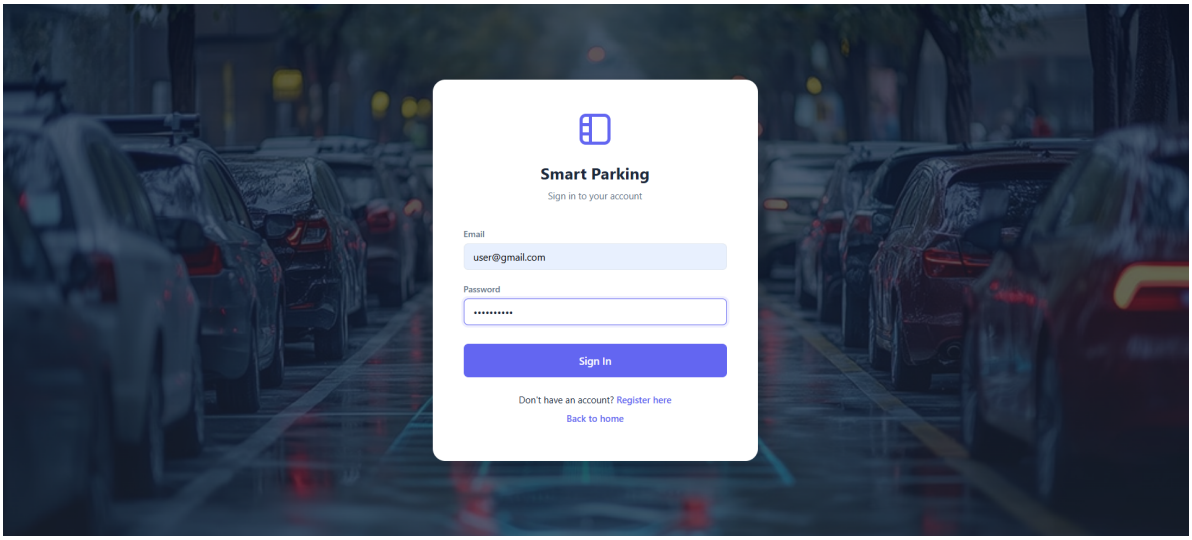


Figure 1: Login Page

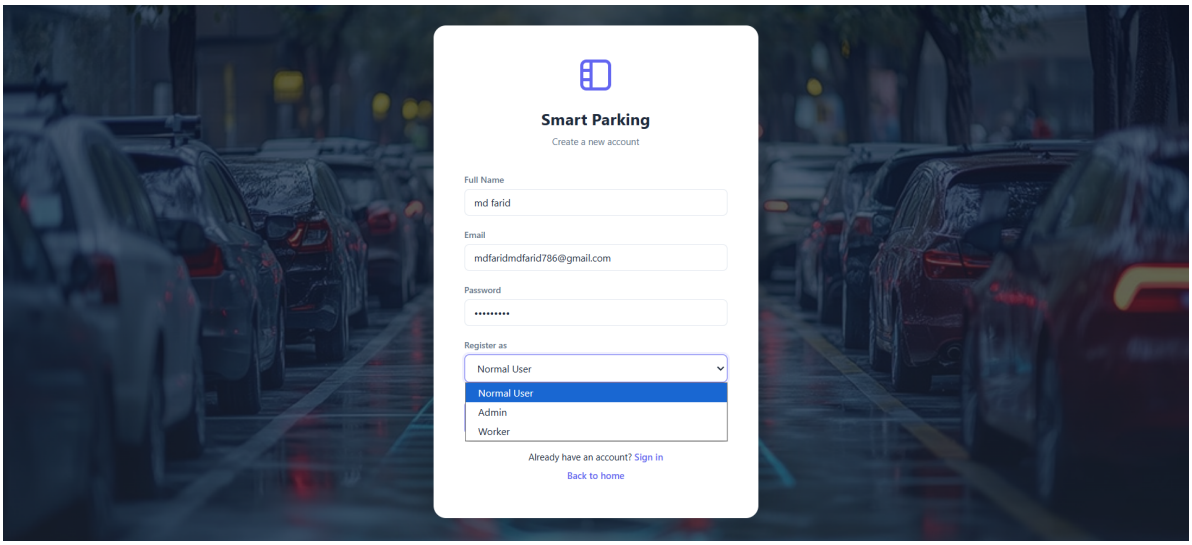


Figure 2: Registration Page

2.5.2 Vehicle Management

What It Does: This feature allows a user to register and manage personal vehicles that will later be used in parking sessions. For the first time, the user only needs to click the car number plate image and complete the vehicle registration process. After registration, the vehicle becomes linked to that user account, and the same vehicle can be selected directly when starting parking in the future.

How to Use It:

1. Open the vehicle management page.
2. Click the car number plate image to add a vehicle for the first time.
3. Enter the plate number and other required details.
4. Save the vehicle and review it in the vehicle list.

5. Use the registered vehicle directly when starting a parking session next time.

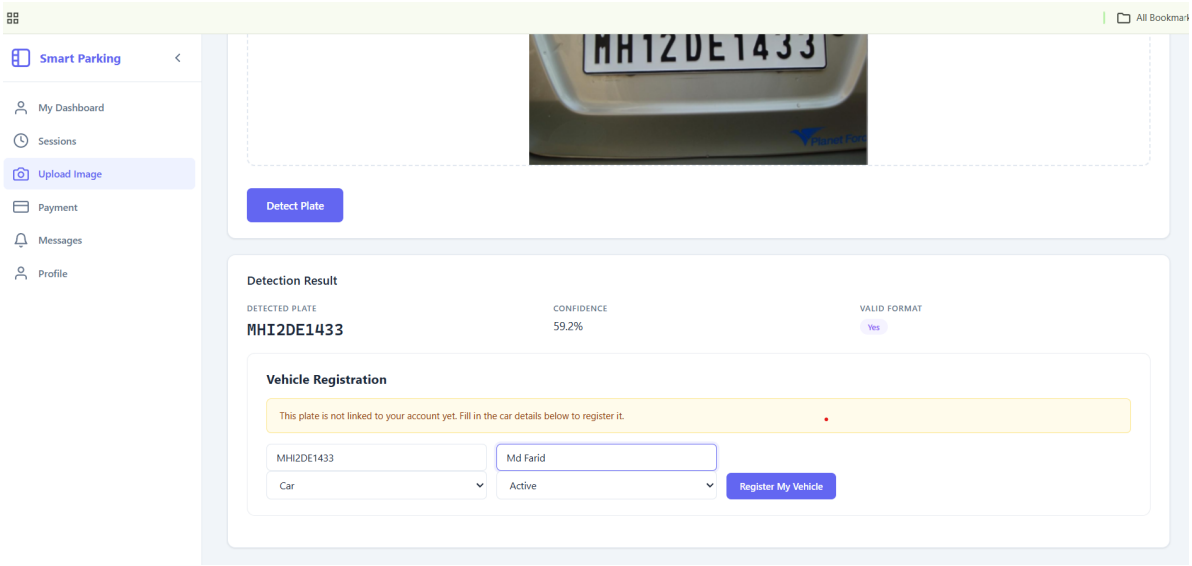


Figure 3: Vehicle Registration Page

2.5.3 Parking Session Creation

What It Does: This feature allows a user to start a parking session by choosing a registered vehicle and the required district or zone. It is the main entry point for active parking use inside the system.

How to Use It:

1. Open the parking session page.
2. Select the required vehicle.
3. Choose the district or parking zone.
4. Confirm the action to start the parking session.

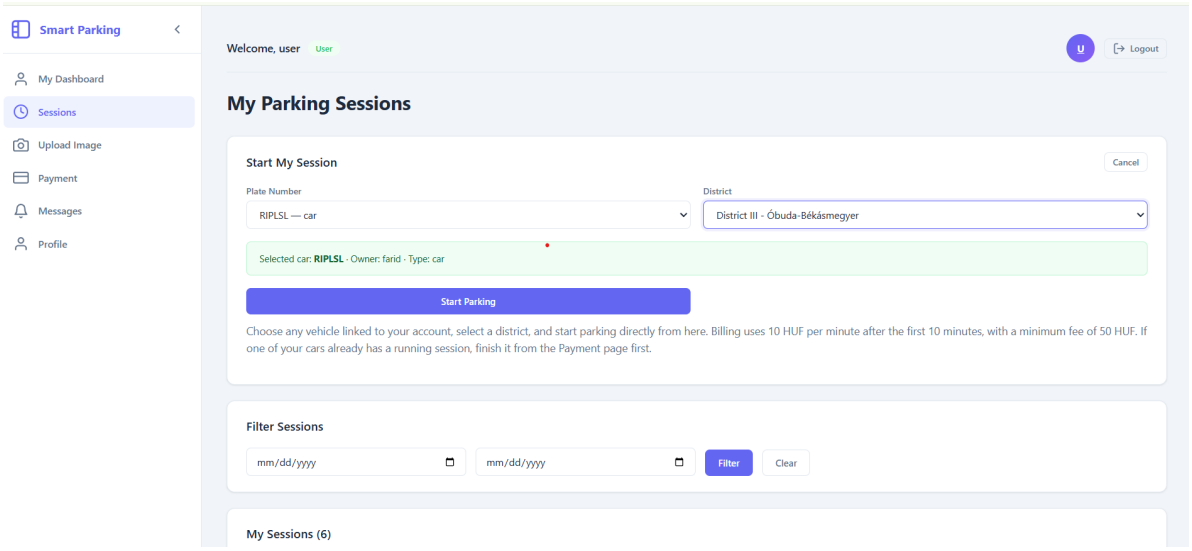


Figure 4: Parking Session Creation Page

2.5.4 Session Tracking and Parking History

What It Does: This feature shows running sessions, completed sessions, and previous parking records. It helps users follow the status of active parking and review earlier activity when needed.

How to Use It:

1. Open the sessions page.
2. Review the active session list if parking is currently running.
3. Open the history area to view completed sessions.
4. Check details such as duration, zone, and related payment information.

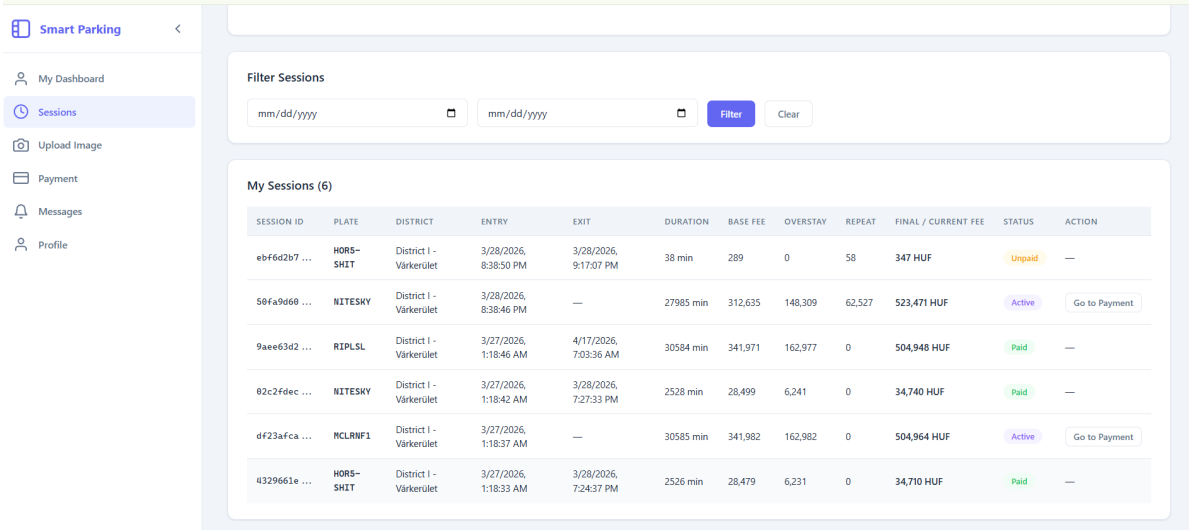


Figure 5: Session Tracking Page

2.5.5 Payment Handling

What It Does: This feature helps users review parking-related fees, unpaid balances, and completed payments. It provides a clearer way to manage financial information connected to parking use.

How to Use It:

1. Open the payment page.
2. Review unpaid sessions, fees, or outstanding balances.
3. Continue to the payment action when required.
4. Confirm the payment and review the updated status.

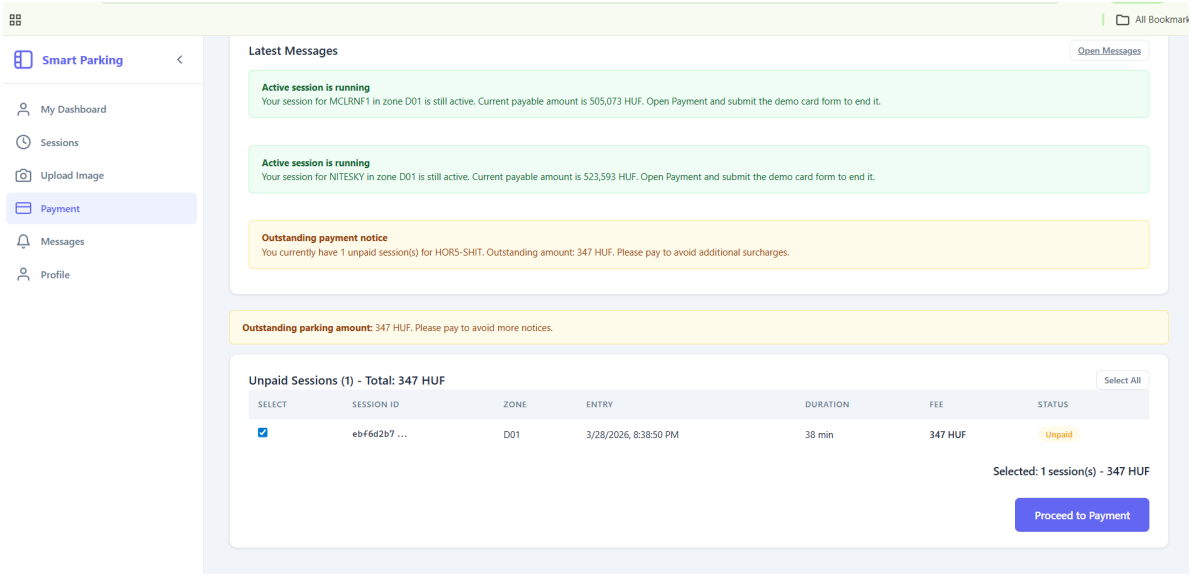


Figure 6: Unpaid Sessions Page

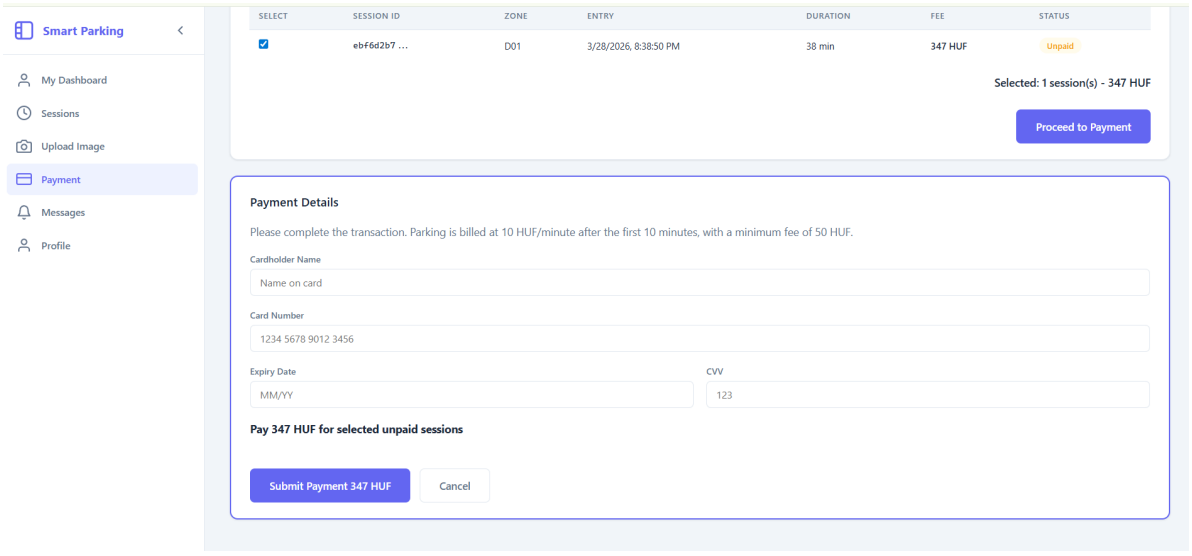


Figure 7: Card Payment Page

2.5.6 Notices and Messages

What It Does: This feature provides users with notices, warnings, and message-based updates related to parking activity. It can be used to communicate important system information, payment-related updates, or enforcement-related messages.

How to Use It:

1. Open the messages page.
2. Review any newly received updates.
3. Open individual items to read their details.
4. Return to the relevant page if follow-up action is required.

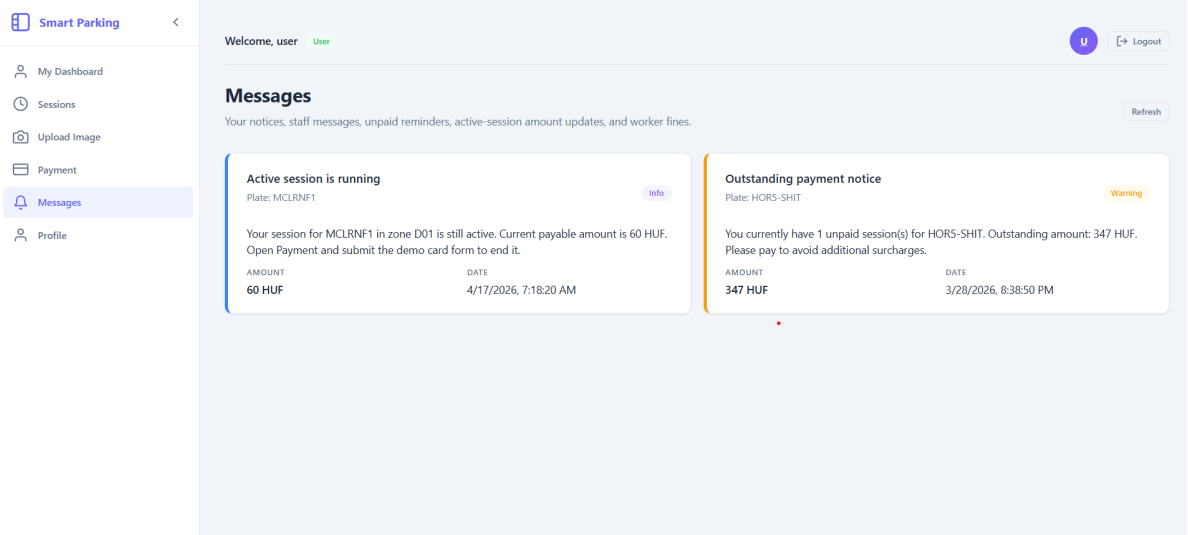


Figure 8: Messages Page

2.5.7 Administrative Dashboard and Reports

What It Does: This feature gives administrators an overview of parking activity, revenue-related information, district operation, and other system-level summaries. It supports supervision and reporting from a central interface.

How to Use It:

1. Sign in with an administrator account.
2. Open the dashboard page.
3. Review the available charts, summaries, and report values.
4. Navigate to related administration pages when additional action is needed.

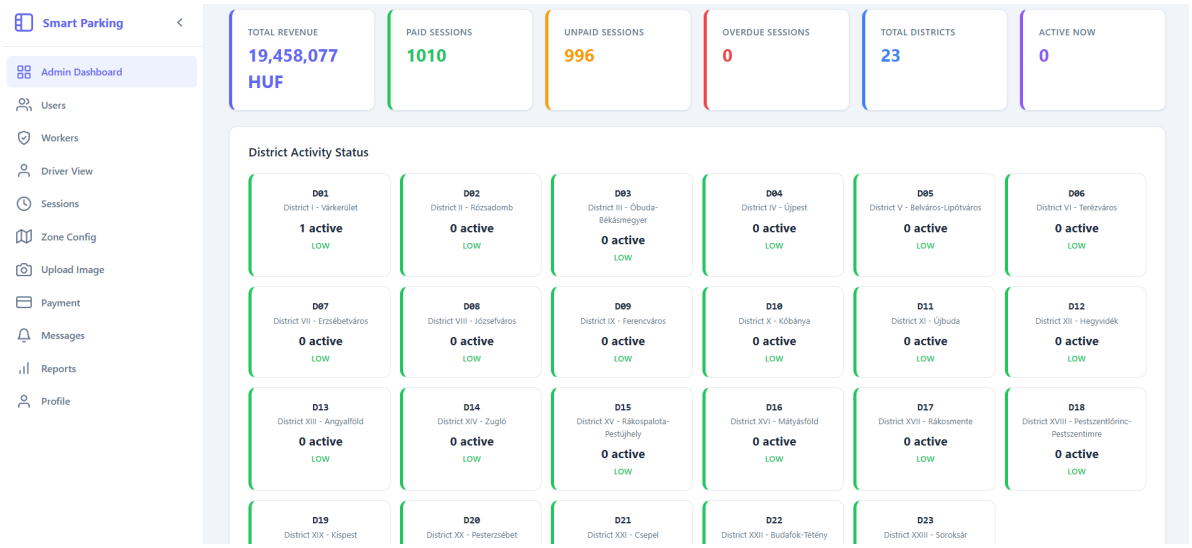


Figure 9: Admin Dashboard Page

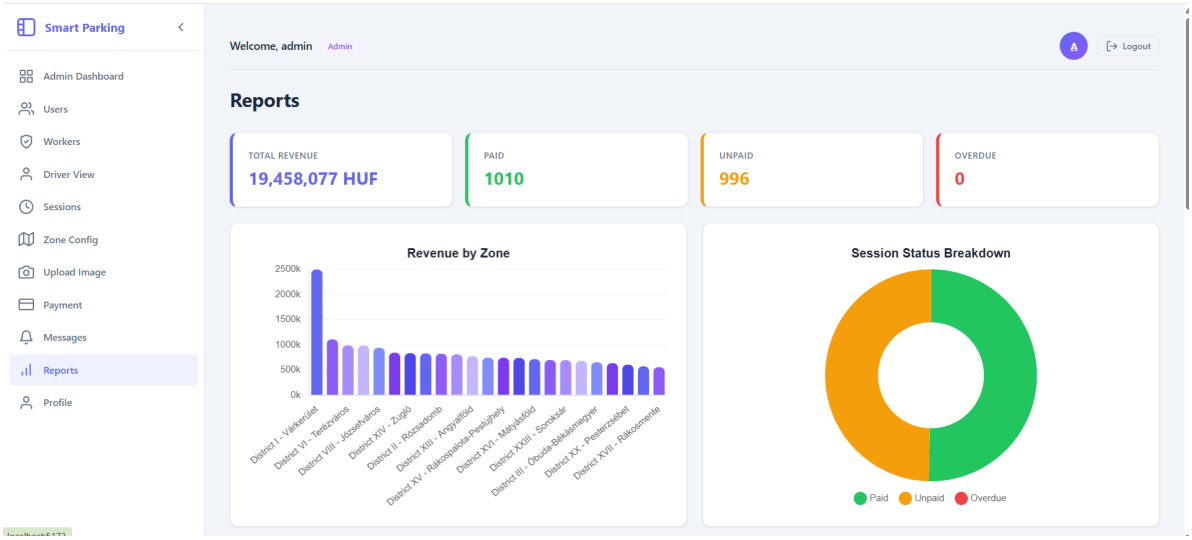


Figure 10: Reports Page

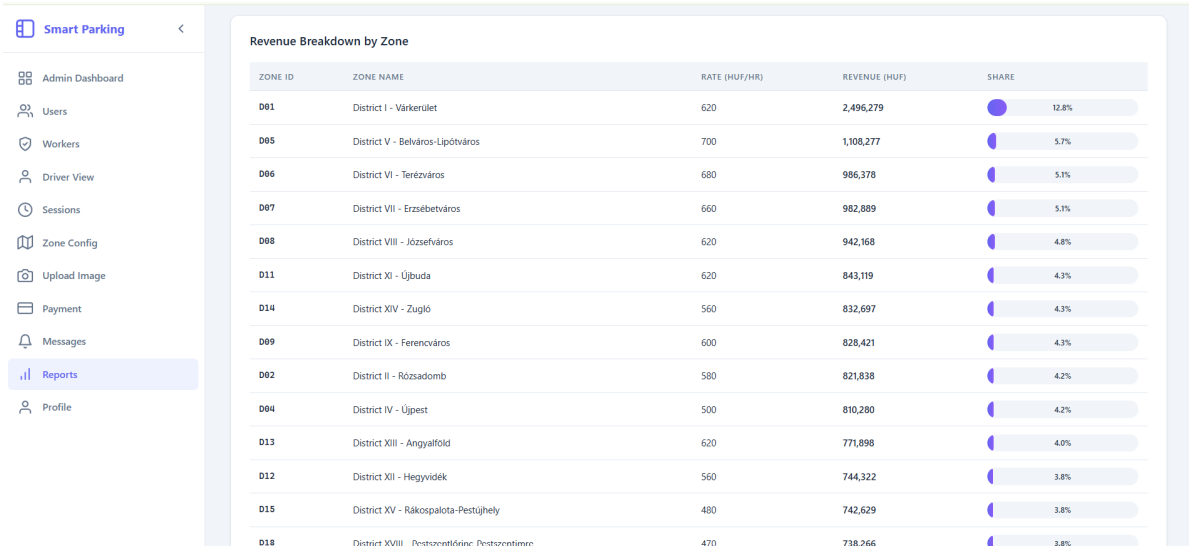


Figure 11: Revenue Breakdown by Zone

2.5.8 District and Zone Management

What It Does: This feature allows the administrator to maintain district-related parking settings and zone information. It is used to organize parking operation according to local district-based structure.

How to Use It:

1. Open the district or zone management page.
2. Review the existing entries.
3. Add, update, or remove the necessary district information.
4. Save the changes and verify the updated configuration.

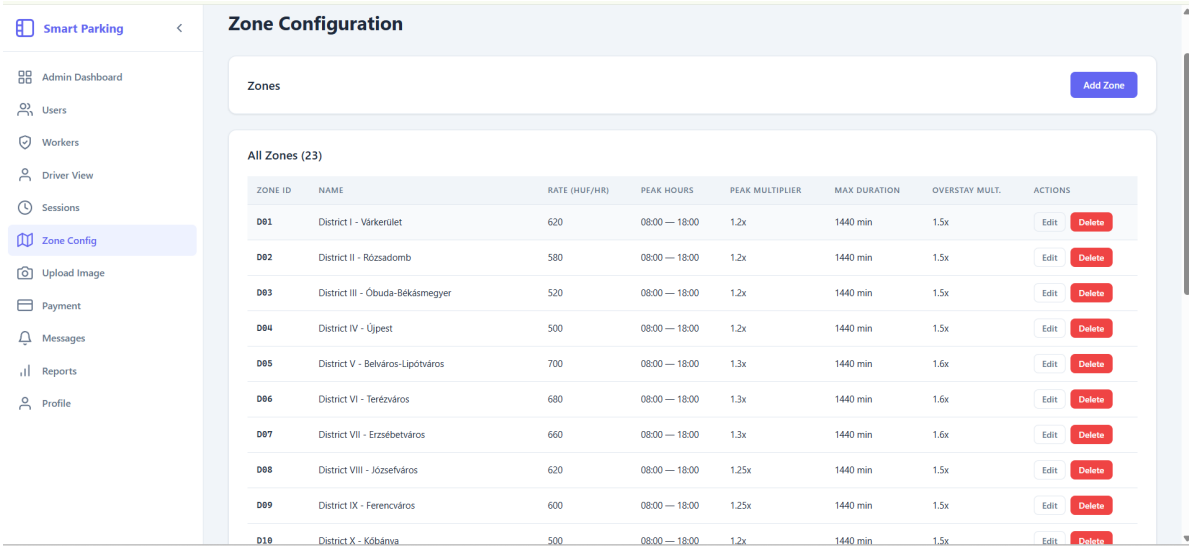


Figure 12: Zone Configuration Page

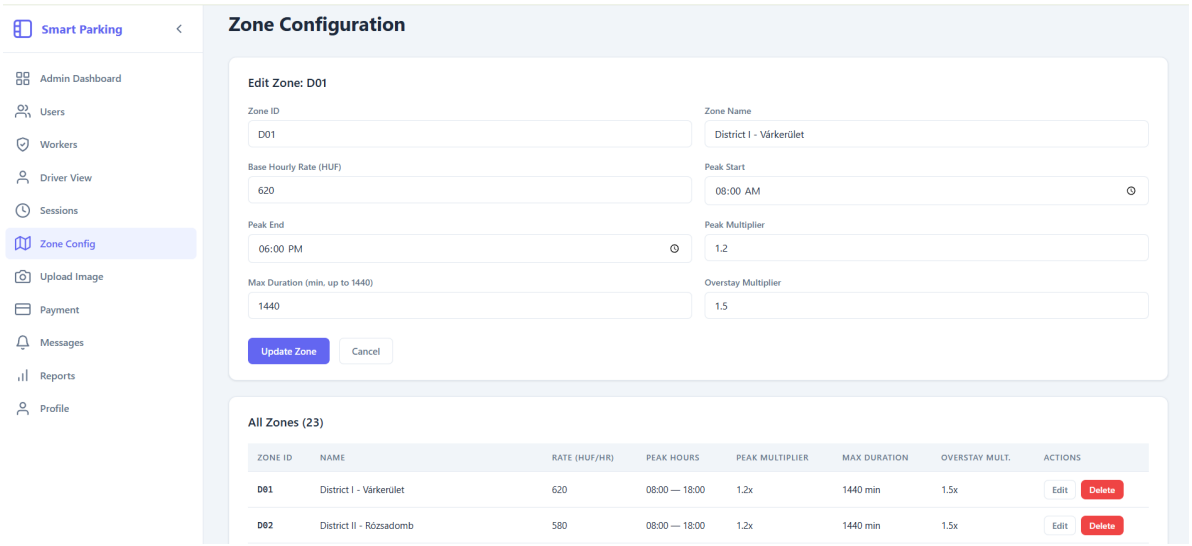


Figure 13: Edit Zone Page

2.5.9 Worker Patrol and Plate Verification

What It Does: This feature supports parking workers during patrol activity by allowing plate checking, session verification, and fine issuance when a vehicle has no active parking session. It also supports upload-based plate recognition through OCR-related tools when needed.

How to Use It:

1. Sign in with a worker account.
2. Open the patrol or verification page.
3. Enter a plate number manually or upload an image for OCR support.
4. Review the returned status and continue with fine issuance if required.

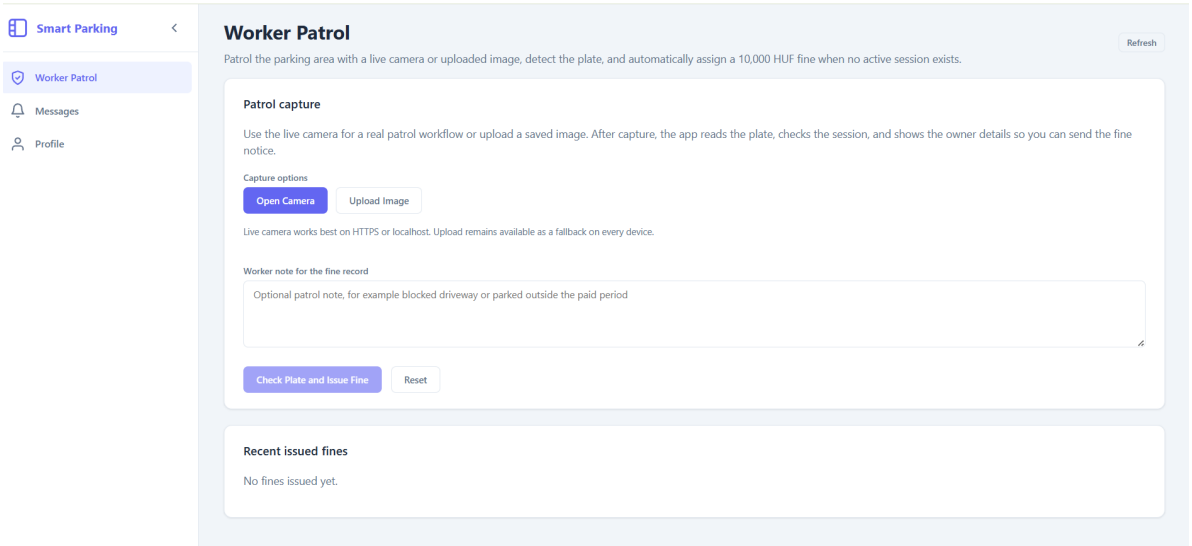


Figure 14: Worker Dashboard Page

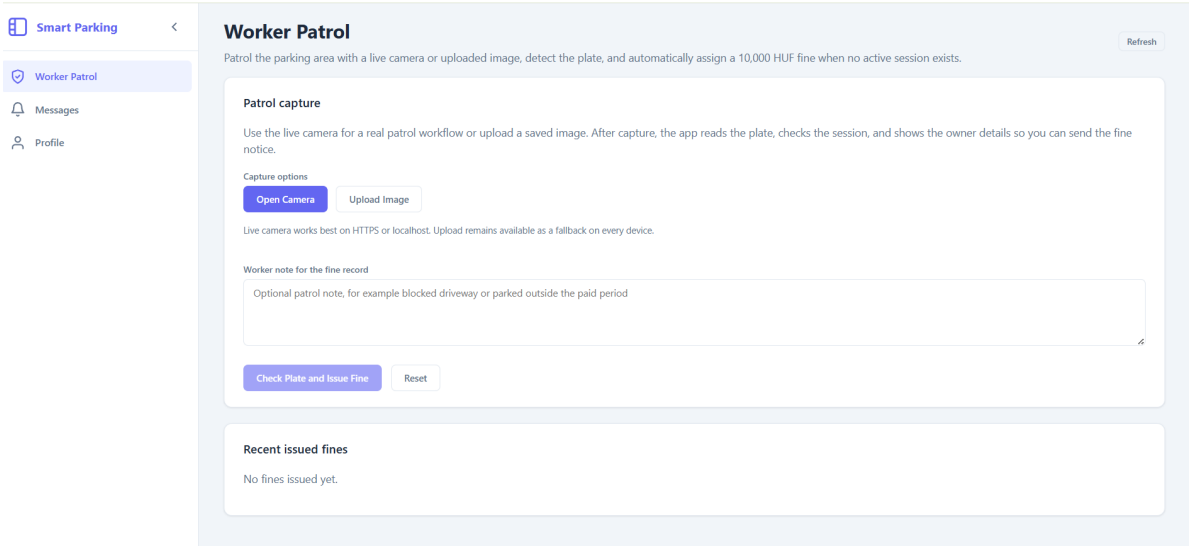


Figure 15: Worker Fine Notice Page

2.5.10 Live Map and Congestion Monitoring

What It Does: This feature provides a live map view that displays parking areas along with traffic congestion information. The map uses different colors to represent parking sessions and congestion levels in each zone, making it easier to understand which areas are busy and which are less occupied.

How to Use It:

1. Open the live map section from the dashboard.
2. View the map with colored zones indicating parking activity and congestion.
3. Identify less congested areas for easier parking.
4. Use the information to decide where to start a parking session.

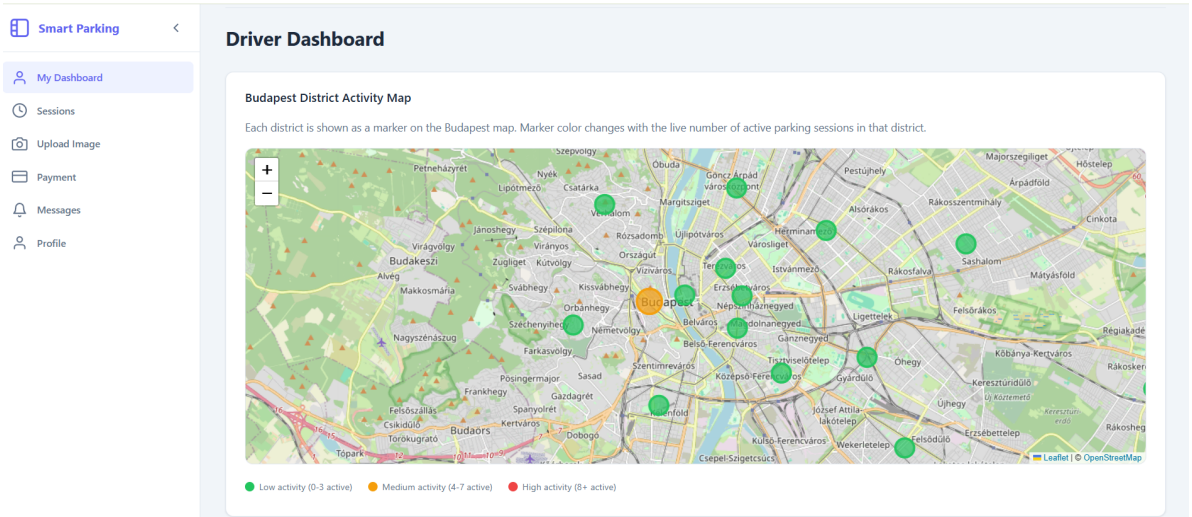


Figure 16: Live Map and Congestion Monitoring Page

Troubleshooting and FAQs

- **Q: I cannot start a parking session. What should I do?**
  A: Make sure that at least one vehicle is registered in your account and that the required district or zone has been selected correctly.

- **Q: Why is my payment status not updating?**
  A: Refresh the payment page and verify that the payment process was completed successfully. If the issue remains, review the related session and notice information.

- **Q: Why does the worker page show that no active session exists?**
  A: Check that the correct plate number was entered or detected. If OCR was used, review the returned plate value before proceeding.

- **Q: Why can I not see admin or worker pages after login?**
  A: The visible interface depends on the assigned role. Make sure you are logged in with the correct account type.

- **Q: The map or dashboard data is not displaying correctly. What should I do?**
  A: Refresh the page, confirm that the backend service is running, and check whether the related district, session, or reporting data has been loaded correctly.

Chapter 3

3. Developer Documentation

3.1 Problem Specification

The Smart Parking Project is a web application designed to support parking management in Budapest through a role-based digital platform. The system combines user-facing parking services, administrative supervision, and worker patrol functions in one environment. It is intended to address common parking management problems such as manual session checking, unclear payment status, slow vehicle verification, and limited visibility of parking activity across districts and zones. The system aims to provide users with the following features:

- **User Authentication and Role Access:** The system allows users to register, log in, and access the platform according to their assigned role.
- **Vehicle Management:** Users can register and manage personal vehicles before starting parking sessions.
- **Parking Session Handling:** The system supports session creation, monitoring, fee review, and session closure.
- **Payment and Notice Management:** Users can review unpaid fees, while the platform also supports notice and fine handling.
- **Administrative Supervision:** Administrators can maintain districts, review records, monitor parking activity, and close sessions when required.
- **Worker Patrol Functions:** Parking workers can check vehicle plates, verify active sessions, and issue fines when a vehicle has no valid session.
- **Live Map and Congestion View:** The platform includes a map-based view that visualizes parking activity and congestion conditions across zones.
- **License Plate Recognition Support:** Image-based plate detection and OCR are used to support faster vehicle verification.

3.2 Tools and Methods Used

The following comprehensive methods, tools, and approaches were integrated to construct the Smart Parking System:

3.2.1 Machine Learning and Image Processing Tools

Plate Recognition / OCR Module:
Used to process uploaded parking images and extract vehicle plate numbers.

Provides automated input handling, reducing manual work and improving operational speed.

Image Validation and Plate Format Checking:
Ensures that recognized plate text follows the expected format before creating a parking session.
Improves reliability and prevents invalid session entries.

3.2.2 APIs and Frameworks

FastAPI:
Fast, lightweight Python framework used to build robust REST APIs.
Handles HTTP requests, routing, request validation, and structured backend communication efficiently.

Pydantic:
Used for request and response validation in backend APIs.
Ensures that data passed between frontend and backend follows the required structure and reduces runtime errors.

3.2.3 Frontend Technologies

React.js:
Component-based frontend technology ensuring an interactive and responsive user experience.

React Router:
Handles navigation between login, dashboards, sessions, profile pages, zone configuration, reports, worker tools, and message pages.

CSS Styling and UI Libraries:
The interface uses regular CSS files together with `react-chartjs-2`, `chart.js`, `leaflet`, and `react-leaflet` for charts and map-based visualization.

Frontend Page-Based Architecture:
Organizes pages such as session management, zone configuration, reports, image upload, admin dashboard, worker dashboard, user dashboards, and payment pages into reusable UI modules.

3.2.4 Development Environment and Tools

Visual Studio Code (VS Code):
Integrated development environment (IDE) supporting efficient coding practices and debugging.

Git and GitHub:
Version control and collaborative project management.

Postman:
API endpoint testing, ensuring correct functionality and performance of backend routes.

3.2.5 Testing and Debugging

Manual API Testing:
API routes were tested using tools such as Postman to verify request handling, response structure, and endpoint behavior.

Browser Developer Tools:
Frontend debugging, network inspection, and interface validation to ensure a responsive user experience.

Component and Workflow Validation:
System workflows such as image upload, session creation, fee calculation, payment updates, and reporting were verified step by step during development.

3.2.6 Backend Architecture and Design Methods

Service Layer Architecture:
Business logic is separated into service classes such as `SessionService`, `ZoneService`, `ReportingService`, `FeeCalculationService`, and `PlateDetectionService`.
This improves maintainability, reusability, and separation of concerns.

Repository Pattern:
Repository classes such as `SessionRepository`, `ZoneRepository`, and `VehicleRepository` are used to isolate database access logic from business logic, while some payment and staff workflows access the database through dedicated API modules.

Layered Backend Design:
The backend is organized into Controllers, Services, and Repositories.
This layered design makes the system easier to test, debug, and extend in future development.

3.2.7 Database and Data Management

SQLite / Relational Database Storage:
Stores parking sessions, vehicles, zones, payments, worker fines, user messages, and zone-related configuration in a structured form.

Structured Entity Modeling:
Core entities such as `User`, `ParkingZone`, `Vehicle`, `ParkingSession`, `Payment`, `WorkerFine`, and `UserMessage` define the internal data structure of the application.

3.3 Key Functionalities

The following functionalities constitute the core features of the Smart Parking System:

1. **Parking Session Management:**
   Automatically creates, updates, and closes parking sessions using vehicle plate recognition and user input. Ensures accurate tracking of parking duration and session status.

2. **Number Plate Recognition:**

Supports image-based vehicle plate detection and recognition, reducing manual data entry and improving operational efficiency.

3. **Dynamic Fee Calculation:**
   Calculates parking fees based on zone configuration, duration of stay, and predefined system parameters.

4. **Multi-Role System Access:**
   Provides different functionalities for Users, Workers, and Admins, ensuring controlled access to system operations.

5. **Zone Management:**
   Allows administrators to add, update, delete, and view parking zones along with their respective configurations.

6. **Payment and Fine Handling:**
   Enables marking sessions as paid, unpaid, or overdue, and supports worker-issued fines and repeat-session penalty tracking.

7. **Reporting and Monitoring:**
   Generates revenue and session-status reports, providing insights into system usage and operational performance.

3.4 Backend Structure and API Endpoints

The backend of the Smart Parking Project is implemented using FastAPI. It exposes route groups for authentication, sessions, zones, vehicles, payments, uploads, and reporting. The following endpoints represent the most important backend functions of the system.

1. User Registration Endpoint (`/auth/register`):

This endpoint creates a new account and assigns the requested role. It also validates the special authorization code when an administrator or worker account is created.

Sample Request:

```
POST /auth/register
{
  "name": "Ali Example",
  "email": "ali@example.com",
  "password": "secret123",
  "role": "user",
  "authorization_code": ""
}
```

2. User Login Endpoint (`/auth/login`):

This endpoint authenticates a user and returns the user identifier, name, email, and role used by the frontend for role-based navigation.

Sample Request:

```
POST /auth/login
{
  "email": "ali@example.com",
  "password": "secret123"
}
```

Sample Response:

```
{
  "user_id": "7f0c...",
  "name": "Ali Example",
  "email": "ali@example.com",
  "role": "user"
}
```

3. Create Parking Session Endpoint (`/sessions/`):

This endpoint starts a new parking session for a selected plate number and parking zone. It can also receive the acting user and role when the session is created from a role-based dashboard.

Sample Request:

```
POST /sessions/
{
  "plate_number": "ABC-1234",
  "zone_id": "Z_001",
  "user_id": "user-001",
  "user_role": "user"
}
```

4. Close Parking Session Endpoint (`/sessions/{session_id}/close`):

This endpoint closes an active parking session, applies the fee-calculation rules, and returns the updated session details.

Sample Request:

```
PUT /sessions/SESSION-001/close
{
  "user_id": "user-001",
  "user_role": "user"
}
```

5. List Parking Sessions Endpoint (`/sessions/`):

This endpoint returns session records and supports optional filtering through query parameters such as date range, role, user identifier, and plate number.

Sample Request:

```
GET /sessions/?date_from=2025-04-01&date_to=2025-04-30&role=admin
```

6. Zone Management Endpoints (`/zones/`):

These endpoints allow the frontend to list, create, update, and delete parking zones used by the Budapest district-based configuration.

Sample Requests:

```
GET /zones/
POST /zones/
PUT /zones/Z_001
DELETE /zones/Z_001
```

7. Vehicle Management Endpoints (`/vehicles/`):

These endpoints list vehicles, return vehicle details, create new vehicle records, and remove vehicles when necessary.

Sample Requests:

```
GET /vehicles/?user_id=user-001&role=user
GET /vehicles/ABC-1234/details?user_id=user-001&role=user
POST /vehicles/
DELETE /vehicles/ABC-1234
```

8. Plate Upload Endpoint (`/upload/plate-image`):

This endpoint receives an uploaded image, processes it through the OCR / detection module, and returns the detected plate text together with confidence and validity information.

Sample Response:

```
{
  "plate_text": "ABC-1234",
  "confidence": 0.95,
  "valid": true
}
```

9. Payment Endpoints (`/payments/pay`, `/payments/checkoutactive`, `/payments-/payall`):

These endpoints support paying selected sessions, checking out an active session directly, and paying all unpaid sessions for a plate number. Additional routes such as `/payments-/notices`, `/payments/adminrecords`, and `/payments/congestion` provide monitoring-oriented data.

10. Reporting Endpoints (`/reports/revenuebyzone`, `/reports/revenuesummary`):
These endpoints return aggregated revenue data and overall payment-status summaries used by the reporting dashboard.

3.5 Frontend Component Structure

The frontend of the Smart Parking System is developed using React and Vite, following a modular and component-based architecture.

Key Components:

1. **Session Management Component:**
   Handles creation, editing, viewing, and closing of parking sessions. Communicates with backend APIs to update session data.

2. **Image Upload Component:**
   Allows users or workers to upload vehicle images for plate recognition.

3. **Authentication Component:**
   Manages user login and logout functionalities and controls access to system features.

4. **Zone Management Component:**
   Enables administrators to manage parking zones, including adding, updating, and deleting zones.

5. **Payment Component:**
   Displays fee details and allows marking sessions as paid, unpaid, or overdue.

6. **Reporting Dashboard:**
   Visualizes system data such as revenue, session status, and zone activity through charts and summary views.

3.6 Development Environment Setup

1. Backend Setup:
Python environment setup (`python -m venv venv`, `pip install -r requirements-ci.txt`).
FastAPI server execution (`uvicorn app.main:app -reload`).
The backend also includes image-processing dependencies such as OpenCV, EasyOCR, and Ultralytics YOLO for license plate detection and recognition.

2. Frontend Setup:
Node.js and npm installation (required for running the frontend application).
Frontend dependency installation (`npm install`).
Configure backend URL or other local settings in the frontend environment file if required.
Frontend application startup (`npm run dev`).

3.7 Image Detection and Processing Logic

The license plate detection logic is implemented in the `PlateDetectionService` class. The two-stage pipeline first uses YOLOv8 to localize the plate region within an uploaded image and then passes the cropped region to EasyOCR for text recognition. The full implementation is shown across the following two figures.

```python
import re

class PlateDetectionService:

    YOLO_CONF_THRESHOLD = 0.25  # threshold set kiya
    CROP_PADDING_PX = 4  # padding value rakha

    def __init__(self):
        import cv2
        import easyocr
        from ultralytics import YOLO

        self.cv2 = cv2
        self.model = YOLO("license_plate.pt")  # model load kiya
        self.reader = easyocr.Reader(['en'], gpu=False)  # OCR setup kiya
        self.plate_pattern = r'^[A-Z0-9-]{5,12}$'  # pattern define kiya

    def _detect_plate_regions(self, image):
        boxes = []
        try:
            yolo_results = self.model(image, verbose=False)  # detection chalaya
        except Exception:
            return boxes  # error handle kiya

        for result in yolo_results:
            if not hasattr(result, "boxes") or result.boxes is None:
                continue
            for box in result.boxes:
                conf = float(box.conf[0]) if hasattr(box, "conf") else 0.0
                if conf < self.YOLO_CONF_THRESHOLD:
                    continue
                x1, y1, x2, y2 = map(int, box.xyxy[0].tolist())
                boxes.append((x1, y1, x2, y2, conf))  # box add kiya

        boxes.sort(key=lambda b: b[4], reverse=True)  # confidence sort kiya
        return boxes

    def _crop_with_padding(self, image, x1, y1, x2, y2):
        h, w = image.shape[:2]
        pad = self.CROP_PADDING_PX  # padding liya
        x1p = max(0, x1 - pad)
        y1p = max(0, y1 - pad)
        x2p = min(w, x2 + pad)
        y2p = min(h, y2 + pad)

        if x2p <= x1p or y2p <= y1p:
            return None  # invalid crop skip
        return image[y1p:y2p, x1p:x2p]

    def _read_best_plate_from_ocr(self, ocr_results):
        best_plate = None
        best_conf = 0.0

        for bbox, text, conf in ocr_results:
            text = text.upper().replace(" ", "").strip()  # text clean kiya
```

Figure 17: Plate detection code

```
            text = text.upper().replace(" ", "").strip()  # text clean kiya
            if re.match(self.plate_pattern, text):
                if conf > best_conf:
                    best_conf = conf
                    best_plate = text  # best plate select

        return best_plate, best_conf

    def process(self, image_bytes: bytes):
        import numpy as np

        np_arr = np.frombuffer(image_bytes, np.uint8)
        image = self.cv2.imdecode(np_arr, self.cv2.IMREAD_COLOR)  # image decode kiya

        if image is None:
            raise ValueError("Invalid image file")

        plate_boxes = self._detect_plate_regions(image)  # plates detect kiye

        best_plate = None
        best_conf = 0.0

        for x1, y1, x2, y2, _det_conf in plate_boxes:
            crop = self._crop_with_padding(image, x1, y1, x2, y2)  # crop banaya
            if crop is None:
                continue

            ocr_results = self.reader.readtext(crop)  # OCR apply kiya
            plate_text, ocr_conf = self._read_best_plate_from_ocr(ocr_results)

            if plate_text and ocr_conf > best_conf:
                best_plate = plate_text
                best_conf = ocr_conf  # best update kiya

        if not best_plate:
            ocr_results = self.reader.readtext(image)  # fallback OCR run
            best_plate, best_conf = self._read_best_plate_from_ocr(ocr_results)

        if not best_plate:
            return {
                "plate_text": None,
                "confidence": 0.0,
                "valid": False,
            }

        return {
            "plate_text": best_plate,
            "confidence": float(best_conf),
            "valid": True,
        }
```

Figure 18: Code continued

3.8 System Advantages

1. **Automated Plate Detection:**
    - The system reduces manual vehicle number entry by using image-based plate

recognition.

2. **Real-Time Parking Operations:**
   - Parking sessions, vehicle entry, and exit operations are handled quickly and efficiently.
3. **Role-Based Access Control:**
   - Separate roles for admin, worker, and user improve security and system organization.
4. **Scalable and Maintainable:**
   - The structured backend and frontend design make the system easier to maintain and extend.
5. **Intuitive User Interface:**
   - A simple and responsive interface makes the system easy to use for different users.
6. **Efficient Data Management:**
   - The system manages vehicles, zones, sessions, and payments in an organized way.

3.9 UML Diagrams

3.9.1 Class Diagram

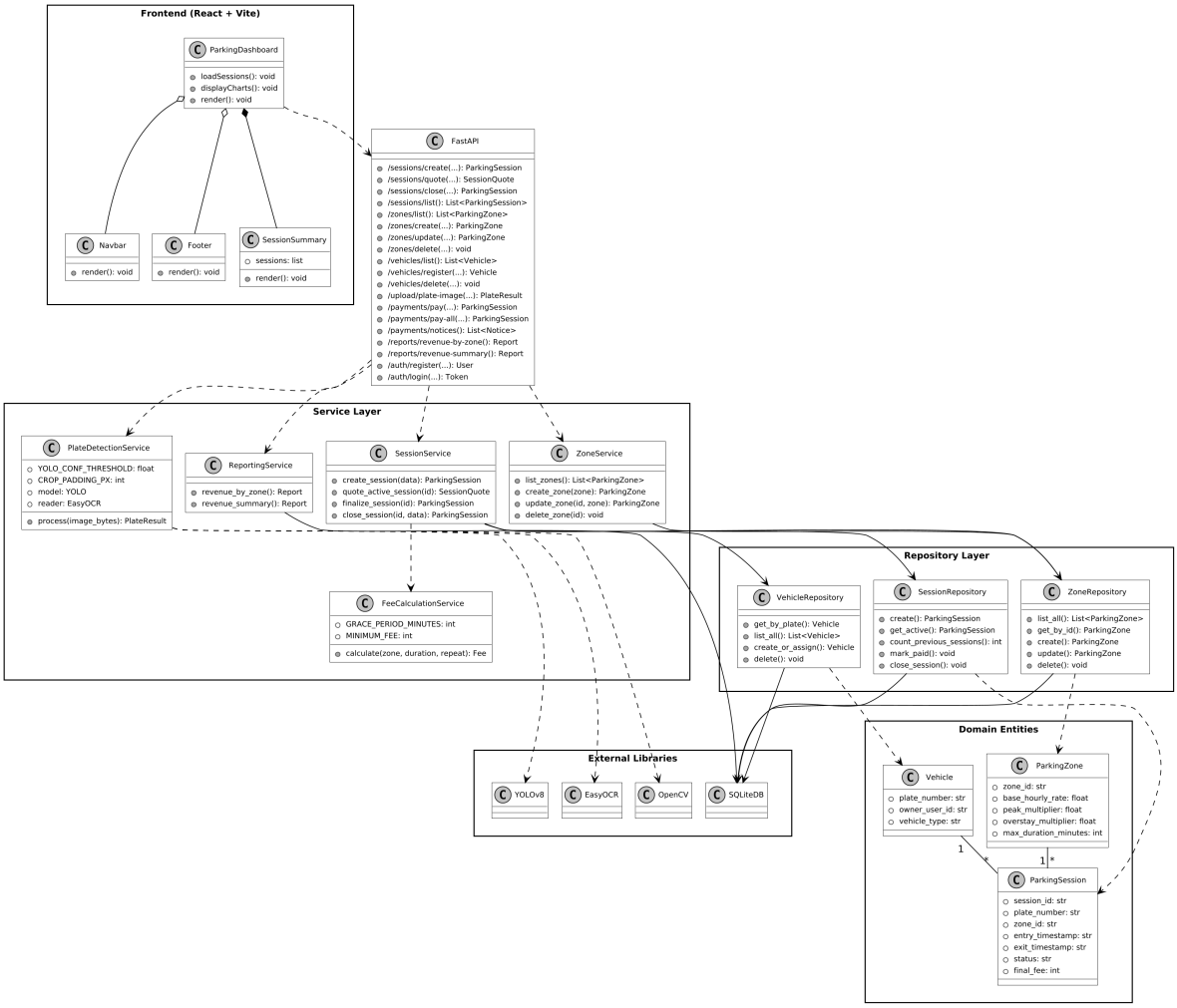


Figure 19: Class Diagram

The following relationships define the structural interactions among various components of the Smart Parking System:

1. **Composition**
   - **ParkingDashboard is composed of SessionSummary.**
     The `SessionSummary` component is closely connected to the `ParkingDashboard` and is used to display summarized session-related information directly inside the main dashboard view.
   - **ParkingSession is associated with Payment generation.**
     A `ParkingSession` may produce a `Payment` record when parking activity is completed and the fee is calculated. This relationship connects the operational parking workflow with the financial part of the system.
2. **Dependency**
   - **ParkingDashboard depends on FastAPI.**
     The frontend dashboard sends API requests to the backend and depends on the

FastAPI layer to retrieve and update parking-related data.

- **SessionController depends on SessionService.**
  The `SessionController` forwards requests related to session creation, closing, and viewing to the service layer, where the main business logic is handled.

- **ZoneController depends on ZoneService.**
  The `ZoneController` relies on the `ZoneService` for adding, updating, deleting, and listing parking zones.

- **Payment routes depend on database-access helpers and session data.**
  The payment modules use stored session information and database queries to complete checkout, pay unpaid sessions, and return notices or monitoring summaries.

- **SessionService depends on PlateDetectionService and FeeCalculationService.**
  The `SessionService` relies on fee-calculation support for determining parking costs and interacts with related session-validation helpers during parking operations.

- **PlateDetectionService depends on YOLOv8 and OCR libraries.**
  The `PlateDetectionService` uses Ultralytics YOLOv8 for plate-region detection and EasyOCR for text recognition, together with OpenCV for image decoding and cropping.

3. **Aggregation**
   - **ParkingDashboard aggregates Navbar and Footer components.**
     The `Navbar` and `Footer` are reusable interface components that support navigation and layout across the frontend. They are used by the dashboard but can also exist independently in other pages.

   - **FastAPI aggregates multiple controllers.**
     The backend routing layer connects with `SessionController`, `ZoneController`, and `PaymentController`, each of which handles a specific part of the parking workflow.

4. **Association**
   - **Vehicle is associated with ParkingSession.**
     A vehicle can be linked with multiple parking sessions, while each parking session belongs to a specific vehicle.

   - **ParkingZone is associated with ParkingSession.**
     Each parking session is assigned to a parking zone, which allows the system to connect parking activity with a district or zone-based structure.

- **Repositories are associated with SQLiteDB.**
  The `SessionRepository`, `ZoneRepository`, and `VehicleRepository` interact with `SQLiteDB` to store and retrieve the corresponding data objects.

- **Repositories are associated with domain entities.**
  The repositories manage the persistence of `ParkingSession`, `ParkingZone`, and `Payment` objects, ensuring that system data remains organized and accessible across operations.

3.9.2 Use Case Diagram

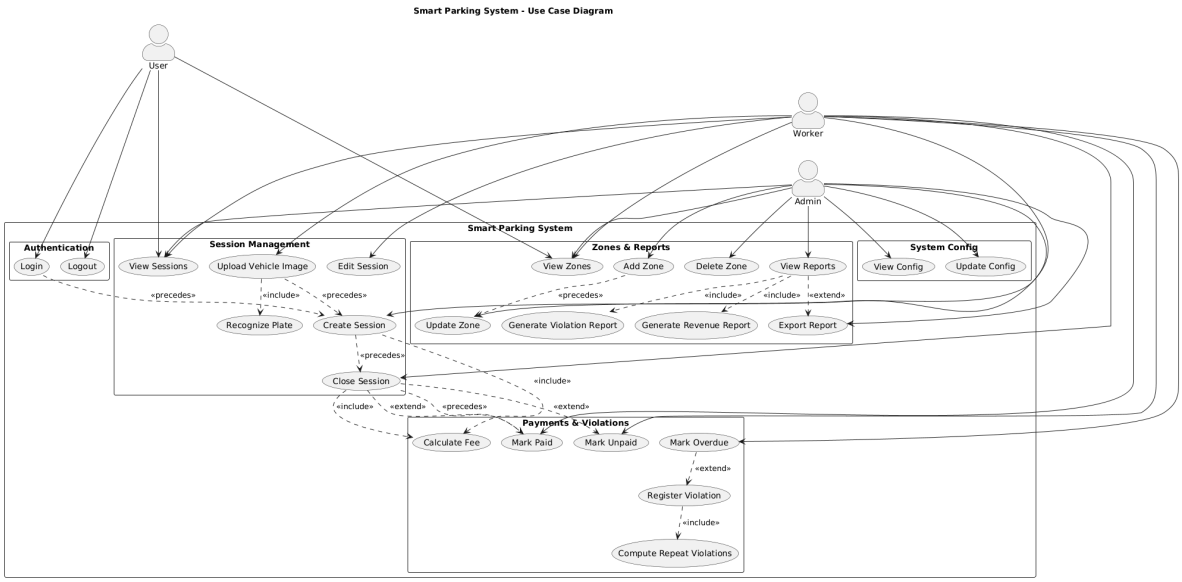


Figure 20: Use Case Diagram

3.9.3 User Sequence Diagram

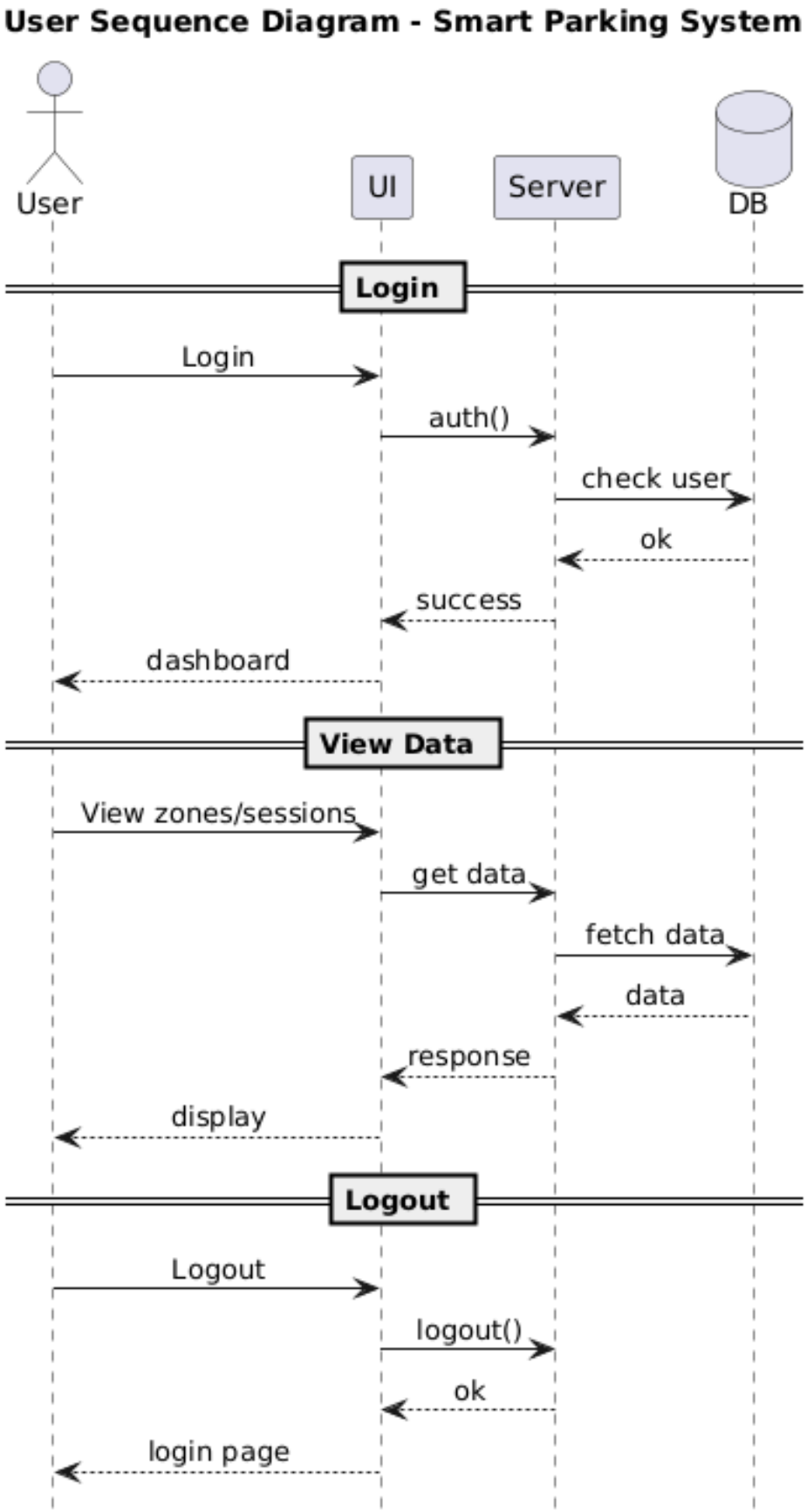


Figure 21: User Sequence Diagram

3.10 Developer Setup

This section provides a step-by-step guide for setting up the development environment for the Smart Parking System, covering prerequisites, backend and frontend configuration, and environment-related setup.

3.10.1 Prerequisites

Before setting up the project, ensure the following tools and packages are installed:

- Python 3.9+ – required to run the FastAPI backend
- pip – Python package manager
- Node.js and npm – required for the frontend (React + Vite)
- Git – for cloning the repository
- Postman (optional) – for testing API endpoints

3.10.2 Clone the Repository

1. Open a terminal or command prompt.
2. Navigate to the directory where you want to clone the project:

```
cd ~/projects
```

3. Clone the Git repository:

```
git clone https://szofttech.inf.elte.hu/gnn/thesis/farid-md-farid.git
cd farid-md-farid
```

3.10.3 Setup Python Backend Environment

1. Navigate to the backend folder:

```
cd backend
```

2. Create a virtual environment:

```
python -m venv venv
```

3. Activate the virtual environment:

```
# Linux/macOS
source venv/bin/activate

# Windows
venv\Scripts\activate
```

4. Install the required Python dependencies:

```
pip install -r requirements-ci.txt
```

5. Start the FastAPI backend server:

```
uvicorn app.main:app --reload
```

3.10.4 Setup Frontend Environment

1. Open a new terminal and navigate to the frontend folder:

```
cd frontend
```

2. Install frontend dependencies:

```
npm install
```

3. Start the frontend development server:

```
npm run dev
```

3.10.5 Configure Environment Variables

The current local development setup does not require mandatory custom environment variables. In development mode, the frontend uses the local backend address automatically, and the backend runs with the default local database file. If needed for deployment or future extension, a `.env` or `.env.local` file can be added to store configuration values such as API base URLs or deployment-specific settings.

3.11 Interactions and Workflow

This section outlines how the main parts of the Smart Parking System interact during normal system usage.

3.11.1 User Interaction Flow

- **Login and Access Control:** The user enters login credentials through the frontend interface. The request is sent to the backend, where the account is verified and the correct role-based dashboard is returned.
- **Parking Session Operations:** A user can register a vehicle, select a parking zone, and start or close a parking session. The frontend sends these requests to the backend, where the session data is validated and stored.
- **Worker Verification Flow:** A worker can enter a plate number manually or upload a vehicle image. The backend checks the session status of the detected plate and returns the result for further action.
- **Administrative Monitoring:** The admin can review zones, users, workers, reports, sessions, and payment-related records through management pages connected to the backend services.

3.11.2 Backend Processing Flow

- The FastAPI backend receives requests from the frontend and validates them using the corresponding schemas.
- The request is forwarded to the appropriate service layer, such as session, vehicle, zone, payment, reporting, or plate-detection services.
- Repository classes interact with the SQLite database to store and retrieve the required records.
- In the image upload workflow, the `PlateDetectionService` decodes the uploaded image, runs YOLOv8 to localize plate regions, applies EasyOCR to the cropped region for text extraction, validates the plate format, and returns the most confident valid result.
- The backend sends structured JSON responses back to the frontend for display and further actions.

3.11.3 Frontend Display and Feedback

- The frontend receives backend responses and updates the interface dynamically according to the returned data.
- Users can view vehicles, sessions, notices, payments, and profile-related information through dedicated pages.
- Workers can view verification results, including detected plate text, confidence value, and session status.
- Administrators can view reports, zone information, worker data, user records, and financial summaries in a structured form.
- The interface is role-based and responsive, allowing the system to present different workflows clearly for user, worker, and admin roles.

3.11.4 Directory Structure

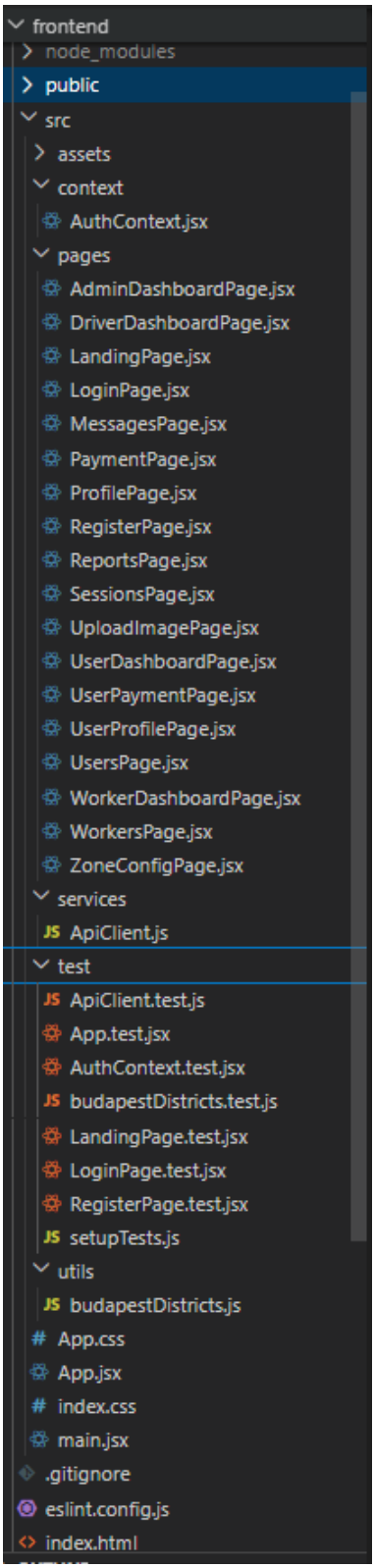


Figure 22: Directory Structure

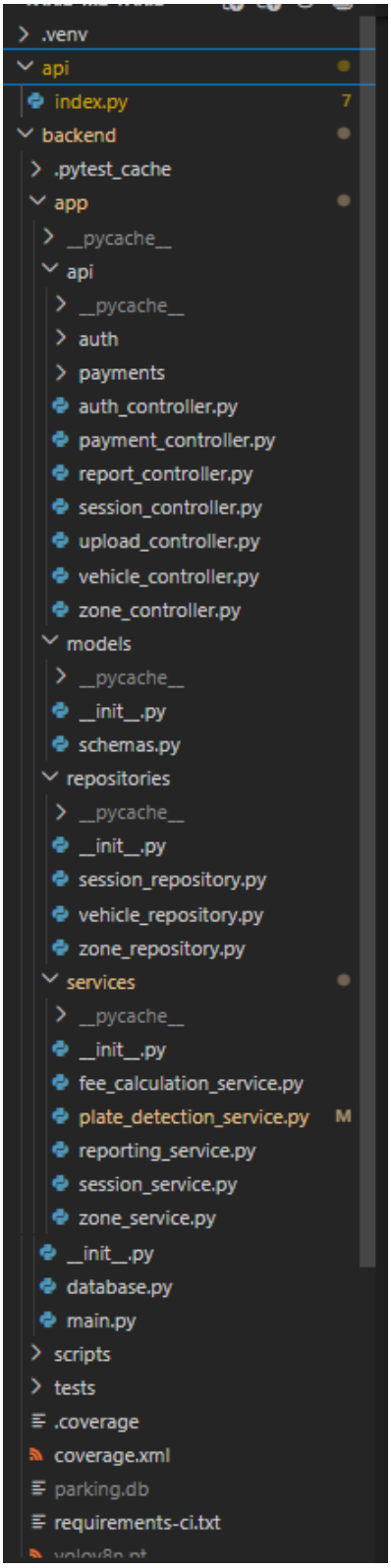


Figure 22: Directory Structure (continued)

3.12 Core Components

This section describes the core modules and functionalities implemented in the Smart Parking System. The system is built using FastAPI for the backend, React and Vite

for the frontend, SQLite for local structured data storage, and OCR-related tools for image-based number plate processing.

3.12.1 Session and Fee Management Module

The `session_service.py` module is one of the most important backend components of the system. It controls the lifecycle of a parking session, starting from session creation and continuing through quote generation, active session monitoring, and final session closure.

- **Vehicle and Zone Validation:** Before a session is created, the service checks whether the selected vehicle exists, whether it is linked to the correct user account, and whether the selected parking zone is valid.
- **Session Creation and Active Session Checks:** The service creates a unique session identifier, stores the entry timestamp, and prevents duplicate active sessions for the same plate number.
- **Fee Calculation Support:** The `FeeCalculationService` calculates the parking amount using zone-based configuration values such as hourly rate, peak-time multiplier, overstay multiplier, grace period, and repeat-session penalty.
- **Session Finalization:** When a session is closed, the backend computes the final parking fee, duration, penalty values, and updated status before saving the completed session record.

3.12.2 FastAPI Server and API Routing

The backend server is implemented in `main.py` using FastAPI. This server acts as the central API layer of the Smart Parking System and connects the different functional modules through dedicated controllers and routers.

- **Router Integration:** The main FastAPI application includes routers for authentication, parking sessions, zones, vehicles, uploads, reports, and payments.
- **Structured API Design:** Controllers such as `session_controller.py`, `zone_controller.py`, `vehicle_controller.py`, `payment_controller.py`, and `upload_controller.py` organize route handling clearly according to system responsibility.
- **Schema Validation:** Request models inside `schemas.py` validate values such as plate numbers, zone identifiers, payment requests, and role-based request fields before the business logic is executed.
- **Frontend Communication:** The backend returns structured JSON responses which are used by the frontend pages to display sessions, fees, payment results, OCR output, and administrative records.

3.12.3 Frontend Interface (React + Vite)

The frontend of the Smart Parking System is implemented using React with Vite. The user interface is page-based and role-oriented, allowing users, workers, and administrators

to access different workflows through a consistent web environment.

- **Role-Based Pages:** The `pages` directory contains dedicated pages such as `UserDashboardPage.jsx`, `WorkerDashboardPage.jsx`, `AdminDashboardPage.jsx`, `SessionsPage.jsx`, `PaymentPage.jsx`, `ReportsPage.jsx`, and `UploadImagePage.jsx`.
- **Authentication State Handling:** The `AuthContext.jsx` module manages login state and role-specific frontend behavior.
- **API Communication Layer:** The `ApiClient.js` file provides the frontend methods used to communicate with backend endpoints for vehicles, sessions, payments, reports, and image upload.
- **Responsive User Experience:** The interface updates dynamically according to backend responses, which allows users to review current sessions, payment states, reports, worker checks, and other parking-related information in a structured way.

3.12.4 Plate Detection and Image Upload Module

The image upload and OCR-related module allows the system to process uploaded vehicle images and extract a likely number plate value. This component is especially useful in worker verification workflows. It follows a two-stage pipeline: a YOLOv8 detector first localizes the plate region, and EasyOCR then reads the text from the cropped region.

- **Upload Endpoint:** The `upload_controller.py` file receives the uploaded image from the frontend through the `/upload/plate-image` endpoint.
- **Plate Detection Service:** The `PlateDetectionService` decodes raw image bytes using OpenCV, runs a license-plate-specific YOLOv8 model to detect plate bounding boxes, crops each detected region, and applies EasyOCR to read the text from the crop. The recognized text is validated against a plate-format regular expression.
- **Stage 1 – Plate Localization:** YOLOv8 returns one or more bounding boxes for plate regions in the uploaded image. Boxes below a configurable confidence threshold are ignored to reduce false positives.
- **Stage 2 – Text Recognition:** For each accepted bounding box, the service crops the plate region (with a small padding) and passes only that crop to EasyOCR. Running OCR on the cropped plate, rather than the whole image, improves recognition accuracy and reduces noise from background text.
- **Graceful Fallback:** If YOLO returns no detections, the service falls back to running EasyOCR on the full image. This ensures the pipeline still produces a result when the detector misses, preserving robustness during real-world use.
- **Confidence-Based Selection:** If multiple OCR results are returned, the service keeps the valid plate with the highest confidence score.

- **Returned Detection Result:** The service returns a structured response containing `plate_text`, `confidence`, and a `valid` flag, which the frontend can display directly to the worker.

3.12.5 Payment, Messaging, and Worker Fine Workflow

The system also contains supporting operational modules for payment handling, user notices, staff messaging, and worker-issued fines. These modules connect the parking workflow with financial and supervisory actions.

- **Payment Processing:** The payment routes support paying selected sessions, checking out an active session directly, and paying all unpaid sessions for a plate number.
- **Notice and Monitoring Support:** The payment-notice module returns unpaid session summaries, threshold-based penalty warnings, and notice information for users and administrators.
- **Worker Fine Logic:** The worker routes verify whether a scanned vehicle already has an active parking session. If no active session exists, the backend can create a worker fine record for the registered vehicle owner.
- **Staff Messaging:** Admins and workers can send direct system messages to users, and the system stores both received and sent messages as part of the account-related data.

3.12.6 Supporting Repositories and Database Layer

The supporting data layer of the project is organized through repositories and a shared database module. This structure separates persistence logic from the service layer and helps keep the backend easier to maintain.

- **Database Access:** The `database.py` module provides the SQLite connection used across the backend.
- **Repository Classes:** Repository modules such as `session_repository.py`, `vehicle_repository.py`, and `zone_repository.py` handle direct data access for the main entities of the application.
- **Structured Data Models:** The project stores users, vehicles, parking sessions, payments, worker fines, messages, and zone-related configuration in a structured relational format.
- **Separation of Concerns:** By dividing the backend into controllers, services, repositories, and schemas, the system becomes easier to test, debug, and extend in future development.

3.13 Testing

To ensure the reliability, correctness, and stability of the Smart Parking System, two key

testing tools were utilized throughout development:

- **Pytest:** Used for validating backend logic, service behavior, repository operations, schema validation, and API workflows.
- **Postman:** Used for endpoint testing, request and response validation, and end-to-end API checking.

In addition to these tools, selected frontend pages and API communication helpers were also checked with Vitest and Testing Library during development. However, the main testing documentation below focuses on the backend and API testing workflow in the same style as the sample documentation.

3.13.1 Testing using Pytest

Pytest was used to perform unit and functional tests on the FastAPI backend. The tested areas include authentication, worker enforcement flows, parking session lifecycle, zone validation, payment processing, reporting, image upload, OCR-based plate recognition, schema validation, and database initialization.

`test_plate_detection_service.py`

- **test_plate_detection_with_yolo_crop()**
  Purpose: Validates the main two-stage pipeline where YOLO detects a plate region and EasyOCR reads text from the cropped image.
  Assertions: Ensures that the service returns the highest-confidence valid plate string from the OCR results on the YOLO crop.
- **test_plate_detection_falls_back_to_whole_image_when_yolo_finds_nothing()**
  Purpose: Validates the graceful fallback path.
  Assertions: Ensures that when YOLO returns no detections, the service falls back to running EasyOCR on the full image and still recognizes a valid plate.
- **test_plate_detection_returns_invalid_when_no_plate_text()**
  Purpose: Validates the no-plate-found path.
  Assertions: Ensures that when neither YOLO nor the fallback OCR produce a plate-shaped string, the service returns `valid: False` with no plate text.
- **test_plate_detection_raises_on_invalid_image()**
  Purpose: Validates input handling for corrupted or unreadable images.
  Assertions: Ensures the service raises `ValueError` when OpenCV fails to decode the uploaded image bytes.
- **test_yolo_low_confidence_box_is_ignored()**
  Purpose: Validates the YOLO confidence threshold filter.
  Assertions: Ensures that YOLO detections below the configured confidence threshold are discarded, and that the fallback OCR path is used instead.

```python
import pytest
from app.services.plate_detection_service import PlateDetectionService

class DummyReader:
    def __init__(self, results):
        self._results = results
    def readtext(self, image):
        return self._results

class DummyCv2:
    IMREAD_COLOR = 1
    def __init__(self, decoded=b"image"):
        self._decoded = decoded
    def imdecode(self, arr, flag):
        return self._decoded

class _Box:
    def __init__(self, x1, y1, x2, y2, conf):
        self.xyxy = [_Tensor([x1,y1,x2,y2])]
        self.conf = [_Scalar(conf)]

class _Tensor:
    def __init__(self, values):
        self._values = values
    def tolist(self):
        return list(self._values)
    def __iter__(self):
        return iter(self._values)

class _Scalar:
    def __init__(self, v):
        self._v = v
    def __float__(self):
        return float(self._v)

class _YoloResult:
    def __init__(self, boxes):
        self.boxes = boxes

class DummyYolo:
    def __init__(self, boxes=None):
        self._boxes = boxes or []
    def __call__(self, image, verbose=False):
        return [_YoloResult(self._boxes)]

class DummyImage:
    def __init__(self, h=480,w=640):
        self.shape=(h,w,3)
    def __getitem__(self,_key):
        return self

_UNSET=object()

def _build_plate_service(ocr_results,yolo_boxes=None,decoded=_UNSET):
    if decoded is _UNSET:
```

Figure 23: Pytest test code

```
54  def _build_plate_service(ocr_results,yolo_boxes=None,decoded=_UNSET):
58      service=PlateDetectionService.__new__(PlateDetectionService)
59      service.cv2=DummyCv2(decoded=decoded)
60      service.reader=DummyReader(ocr_results)
61      service.model=DummyYolo(boxes=yolo_boxes)
62      service.plate_pattern=r"^[A-Z0-9-]{5,12}$"
63      return service
64
65  def test_plate_detection_with_yolo_crop():
66      service=_build_plate_service(
67          ocr_results=[
68              (None,"abc 123",0.65),
69              (None,"A1",0.99),
70              (None,"XYZ-999",0.91),
71          ],
72          yolo_boxes=[_Box(50,100,250,160,0.88)],
73      )
74      result=service.process(b"image-bytes")
75      assert result=={"plate_text":"XYZ-999","confidence":0.91,"valid":True}
76
77  def test_plate_detection_falls_back_to_whole_image_when_yolo_finds_nothing():
78      service=_build_plate_service(
79          ocr_results=[(None,"BUD-2025",0.87)],
80          yolo_boxes=[],
81      )
82      result=service.process(b"image-bytes")
83      assert result=={"plate_text":"BUD-2025","confidence":0.87,"valid":True}
84
85  def test_plate_detection_returns_invalid_when_no_plate_text():
86      service=_build_plate_service(
87          ocr_results=[(None,"bad",0.7)],
88          yolo_boxes=[],
89      )
90      assert service.process(b"image")=={
91          "plate_text":None,
92          "confidence":0.0,
93          "valid":False,
94      }
95
96  def test_plate_detection_raises_on_invalid_image():
97      service=_build_plate_service(
98          ocr_results=[],
99          yolo_boxes=[],
100         decoded=None,
101     )
102     with pytest.raises(ValueError,match="Invalid image file"):
103         service.process(b"broken")
104
105 def test_yolo_low_confidence_box_is_ignored():
106     service=_build_plate_service(
107         ocr_results=[(None,"ABC-1234",0.93)],
108         yolo_boxes=[_Box(0,0,100,50,0.10)],
109     )
110     result=service.process(b"image")
111     assert result=={"plate_text":"ABC-1234","confidence":0.93,"valid":True}
```

Figure 24: Test code continued

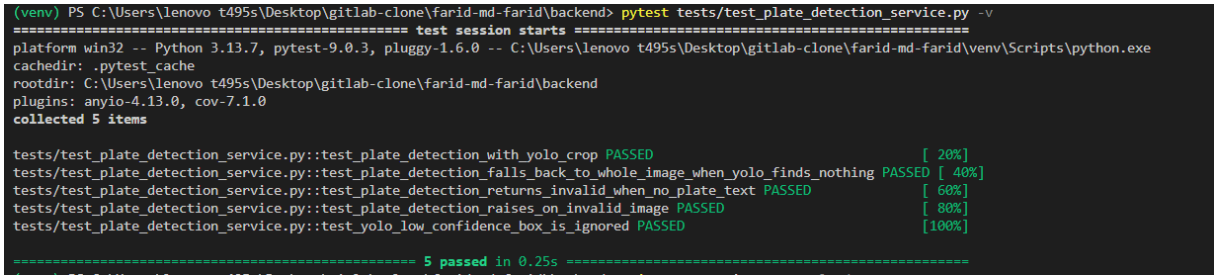


Figure 25: Pytest result

`test_upload_api.py`

- **test_upload_plate_image_endpoint(client, monkeypatch)**
  Purpose: Tests the upload endpoint used for plate-image submission.
  Assertions: Confirms that the uploaded image is accepted correctly and that the API returns a plate text, confidence value, and valid status.

`test_session_service.py`

- **test_session_service_quote_create_finalize_close_and_enrich(seed_data)**
  Purpose: Validates the complete lifecycle of parking sessions.
  Assertions: Verifies ownership checks, quote generation, session creation, fee finalization, role-based close permissions, and validation of invalid timestamps or missing sessions.

`test_zone_service.py`

- **test_zone_service_validation_and_crud(seed_data)**
  Purpose: Tests zone validation together with create, read, update, and delete operations.
  Assertions: Ensures valid zones are stored correctly and invalid zone configurations are rejected.

`test_fee_calculation_service.py`

- **test_fee_calculation_helpers_and_paths()**
  Purpose: Verifies the helper methods and fee calculation rules used during parking checkout.
  Assertions: Confirms minimum-fee handling, peak-hour detection, overstay charges, repeat penalties, and the correctness of the final fee formula.

`test_auth_accounts_api.py`

- **test_auth_register_login_users_and_summary(client, seed_data)**
  Purpose: Tests registration, login, user listing, and summary retrieval.
  Assertions: Checks role-based registration rules, duplicate-account rejection, login success and failure cases, and the correctness of returned user statistics.

`test_auth_staff_api.py`

- **test_staff_messages_worker_fines_and_listing(client, seed_data)**

Purpose: Validates admin messaging and worker fine workflows.
Assertions: Ensures only authorized roles can send messages, verifies worker scan-and-fine behavior, checks duplicate fine prevention, and confirms fine-notification delivery.

`test_vehicle_api.py`

- **test_vehicle_endpoints(client, seed_data)**
  Purpose: Tests vehicle listing, lookup, detail retrieval, creation, and deletion.
  Assertions: Confirms role-based visibility, owner-restricted access, summary generation, plate normalization, duplicate prevention, and deletion behavior.

`test_payments_transactions_api.py`

- **test_payment_transaction_endpoints(client, seed_data)**
  Purpose: Tests payment-related transaction endpoints.
  Assertions: Verifies single-session payment, active-session checkout, payment history retrieval, unpaid-session lookup, penalty checks, and full payment by plate.

`test_payments_notices_api.py`

- **test_payment_notices_and_monitoring_endpoints(client, seed_data)**
  Purpose: Tests payment notices, admin records, congestion data, and penalty-related monitoring.
  Assertions: Ensures generated notices are returned correctly for users and admins, confirms admin payment records, and validates congestion summaries.

`test_report_api.py` and `test_reporting_service.py`

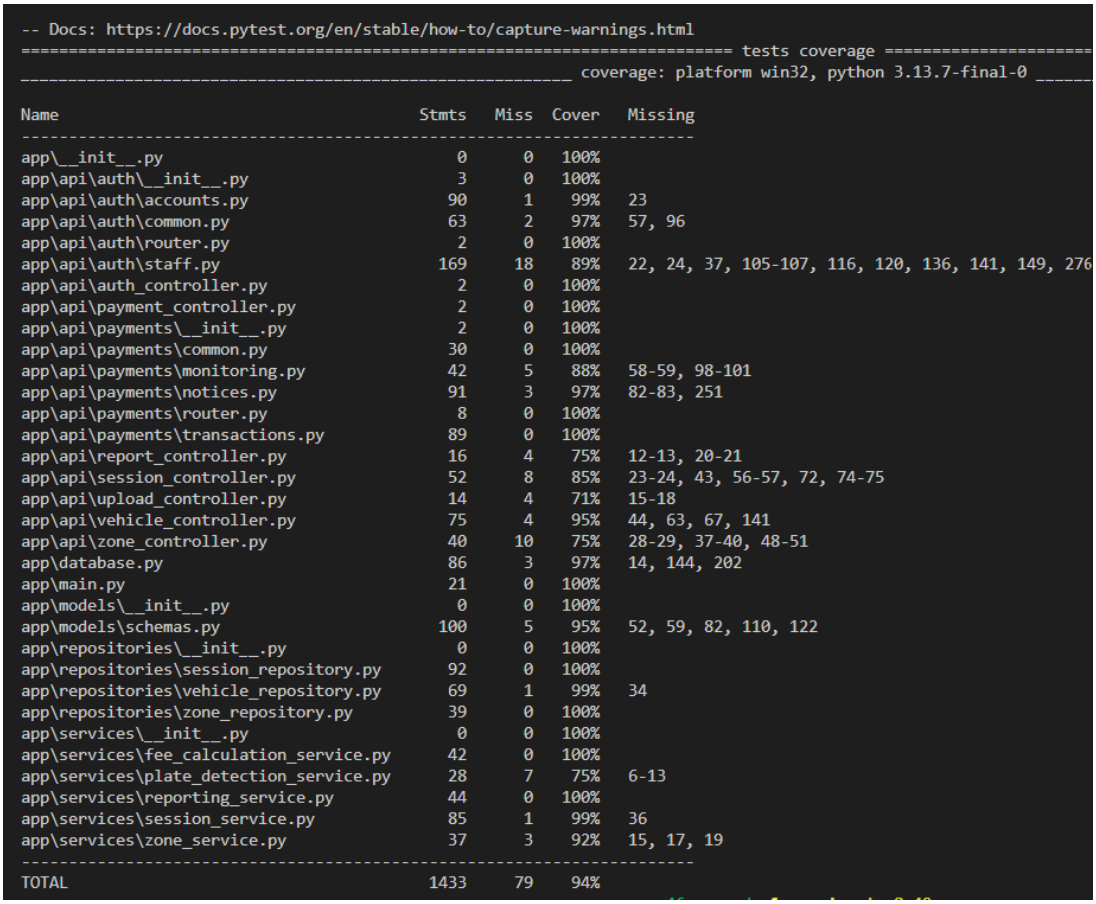
- **Purpose:** Validate reporting endpoints and reporting-service summaries.
  Assertions: Confirm revenue-by-zone output, paid/unpaid/overdue counts, total revenue summary, and filtered session reporting by date range and role.

`test_database.py` and `test_schemas.py`

- **Purpose:** Validate database initialization, seed data, and request schema rules.
  Assertions: Ensure the database is created correctly, Budapest zones and admin data are seeded, and invalid schema input is rejected through validation.

`test_health_api.py`

- **Purpose:** Checks whether the health endpoint responds correctly for basic service monitoring.
  Assertions: Confirms that the backend reports a healthy status when the application is running.



```
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
================================================================================ tests coverage ================================
_____________________________________________________ coverage: platform win32, python 3.13.7-final-0 ______

Name                                          Stmts   Miss  Cover   Missing
---------------------------------------------------------------------------
app\__init__.py                                   0      0   100%
app\api\auth\__init__.py                          3      0   100%
app\api\auth\accounts.py                         90      1    99%   23
app\api\auth\common.py                           63      2    97%   57, 96
app\api\auth\router.py                            2      0   100%
app\api\auth\staff.py                           169     18    89%   22, 24, 37, 105-107, 116, 120, 136, 141, 149, 276
app\api\auth_controller.py                        2      0   100%
app\api\payment_controller.py                     2      0   100%
app\api\payments\__init__.py                      2      0   100%
app\api\payments\common.py                       30      0   100%
app\api\payments\monitoring.py                   42      5    88%   58-59, 98-101
app\api\payments\notices.py                      91      3    97%   82-83, 251
app\api\payments\router.py                        8      0   100%
app\api\payments\transactions.py                 89      0   100%
app\api\report_controller.py                     16      4    75%   12-13, 20-21
app\api\session_controller.py                    52      8    85%   23-24, 43, 56-57, 72, 74-75
app\api\upload_controller.py                     14      4    71%   15-18
app\api\vehicle_controller.py                    75      4    95%   44, 63, 67, 141
app\api\zone_controller.py                       40     10    75%   28-29, 37-40, 48-51
app\database.py                                  86      3    97%   14, 144, 202
app\main.py                                      21      0   100%
app\models\__init__.py                            0      0   100%
app\models\schemas.py                           100      5    95%   52, 59, 82, 110, 122
app\repositories\__init__.py                      0      0   100%
app\repositories\session_repository.py           92      0   100%
app\repositories\vehicle_repository.py           69      1    99%   34
app\repositories\zone_repository.py              39      0   100%
app\services\__init__.py                          0      0   100%
app\services\fee_calculation_service.py          42      0   100%
app\services\plate_detection_service.py          28      7    75%   6-13
app\services\reporting_service.py                44      0   100%
app\services\session_service.py                  85      1    99%   36
app\services\zone_service.py                     37      3    92%   15, 17, 19
---------------------------------------------------------------------------
TOTAL                                          1433     79    94%
```

Figure 26: Total coverage

3.13.2 API Testing Using Postman

Postman was used extensively during development to test and verify API functionality, including input and output behavior, response structure, validation errors, and endpoint integration. It was particularly useful for checking the communication between the React frontend and the FastAPI backend.

Prerequisites

- **API Running Locally:** FastAPI server running at `http://127.0.0.1:8000`
- **Postman Installed:** Postman desktop application available for sending requests
- **Database Ready:** Local database initialized with the required schema and sample data

Base URL

`http://127.0.0.1:8000`

3.13.2.1 Authentication Endpoints 1. Register User

- **Endpoint:** `/auth/register`
- **Method:** POST

- **Headers:** `Content-Type: application/json`

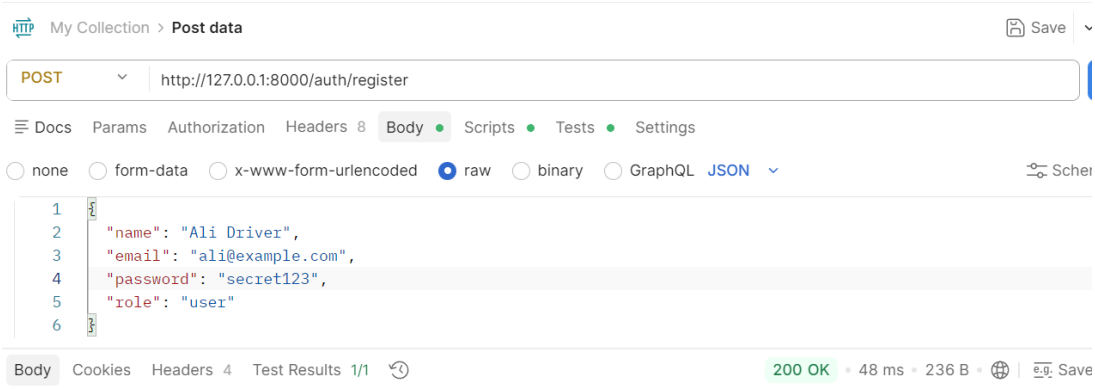


Figure 27: User Registration Test

2. Login User

- **Endpoint:** `/auth/login`
- **Method:** POST
- **Headers:** `Content-Type: application/json`

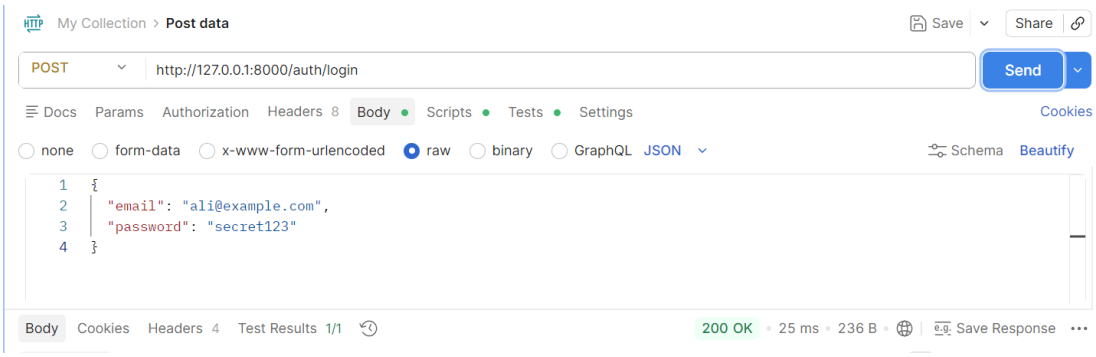


Figure 28: User Login Test

3.13.2.2 Parking and Upload Endpoints 1. Create Parking Session

- **Endpoint:** `/sessions/`
- **Method:** POST
- **Headers:** `Content-Type: application/json`
- **Body:**

```
{
  "plate_number": "ABC-123",
  "zone_id": "D01",
  "user_id": "user-001",
  "user_role": "user"
```

}

- **Expected Response:** Newly created session with active status.

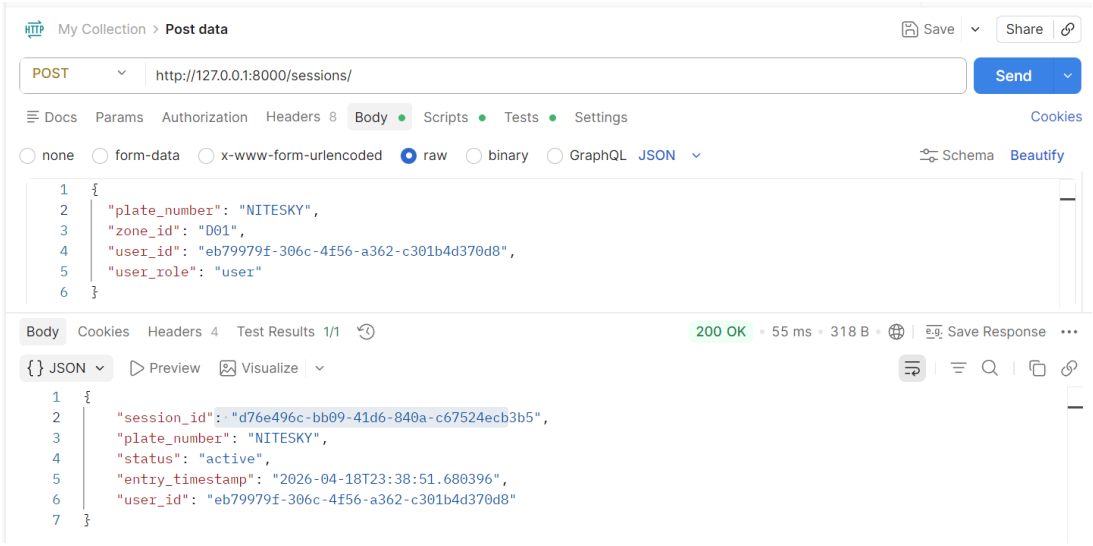


Figure 29: Parking Session Creation Test

2. Upload Plate Image

- **Endpoint:** `/upload/plate-image`
- **Method:** POST
- **Body Type:** `form-data`
- **Field:** `file` = image file
- **Expected Response:** JSON response containing `plate_text`, `confidence`, and `valid`.

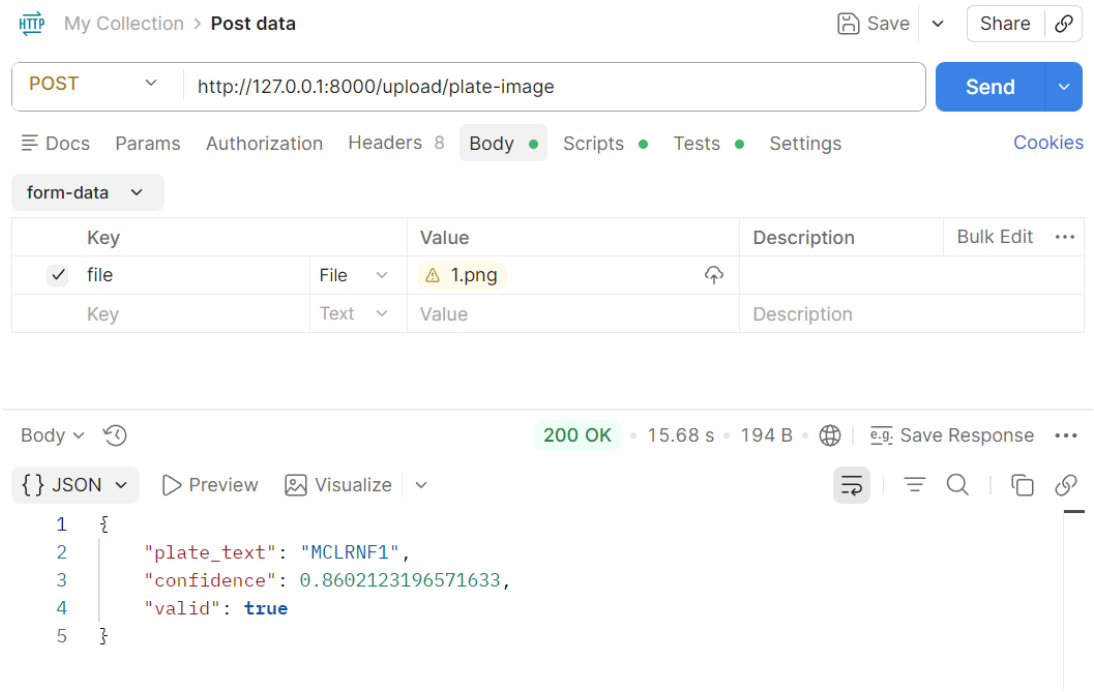


Figure 30: Plate Image Upload Test

3.13.2.3 Zone, Payment, and Report Endpoints 1. Zone Management Test

- **Endpoint:** `/zones/`
- **Method:** POST
- **Headers:** `Content-Type: application/json`
- **Expected Response:** Created zone details returned after validation.

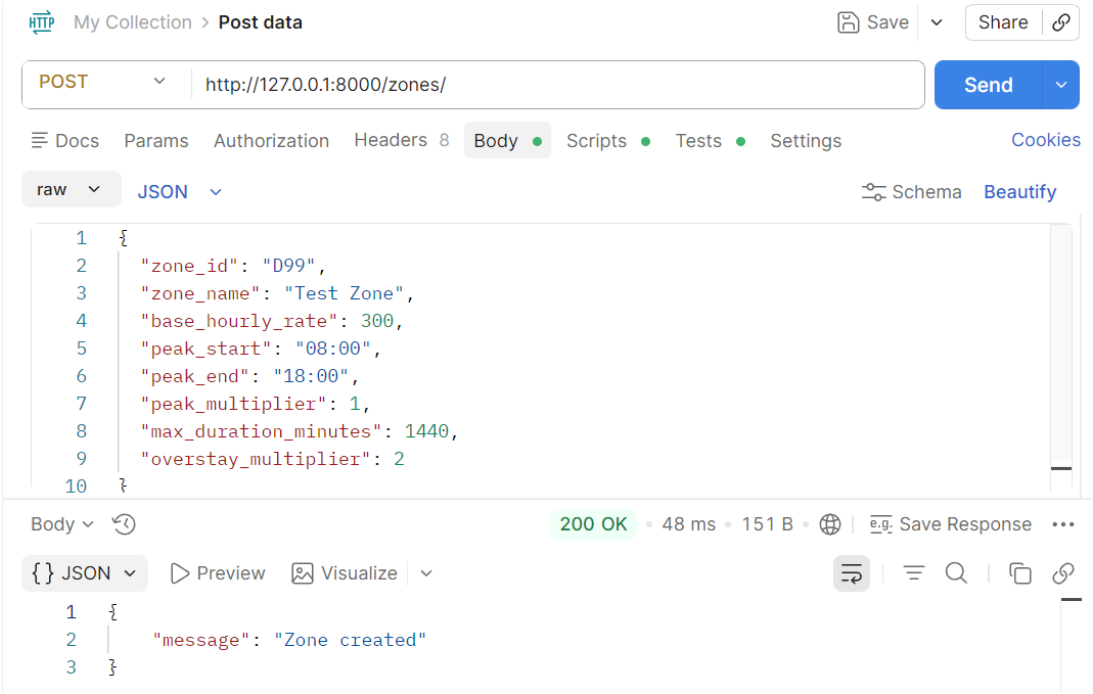


Figure 31: Zone Management Test

2. Payment Endpoint Test

- **Endpoint:** `/payments/pay`
- **Method:** POST
- **Headers:** `Content-Type: application/json`
- **Expected Response:** Payment summary with paid amount and receipt list.

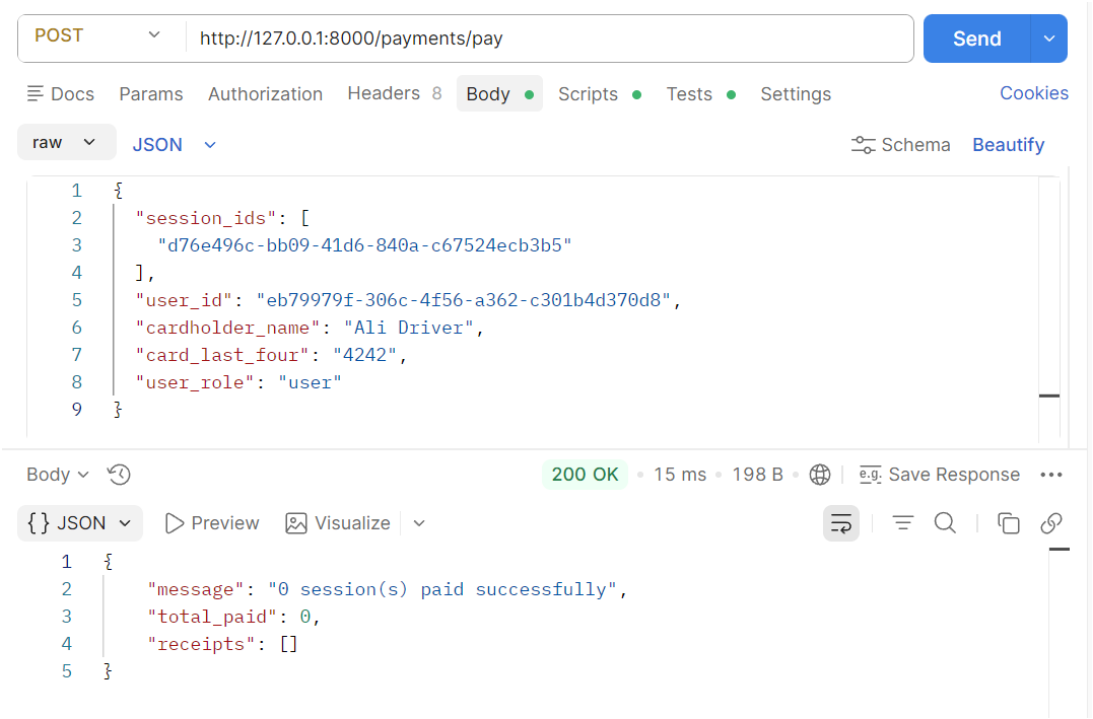


Figure 32: Payment Test

3. Report Endpoint Test

- **Endpoint:** `/reports/revenue-summary`
- **Method:** GET
- **Expected Response:** Aggregated revenue and status counters.

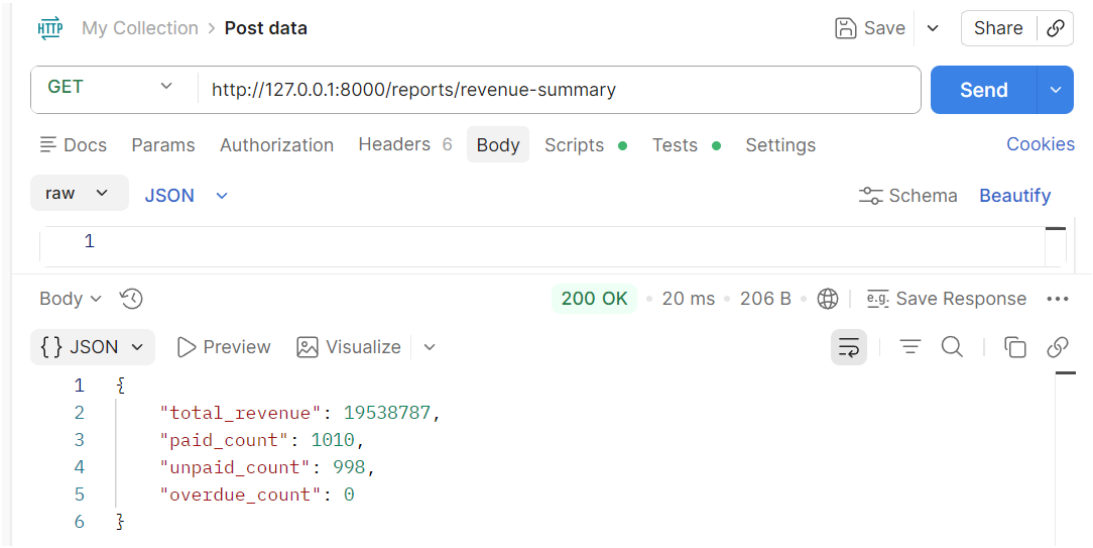


Figure 33: Revenue Summary Test

3.13.2.4 Error and Validation Tests 1. Invalid Login Test

- **Body:**

```
{
  "email": "ali@example.com",
  "password": "wrong-password"
}
```

- **Expected Response:** Authentication failure message with the corresponding error status.

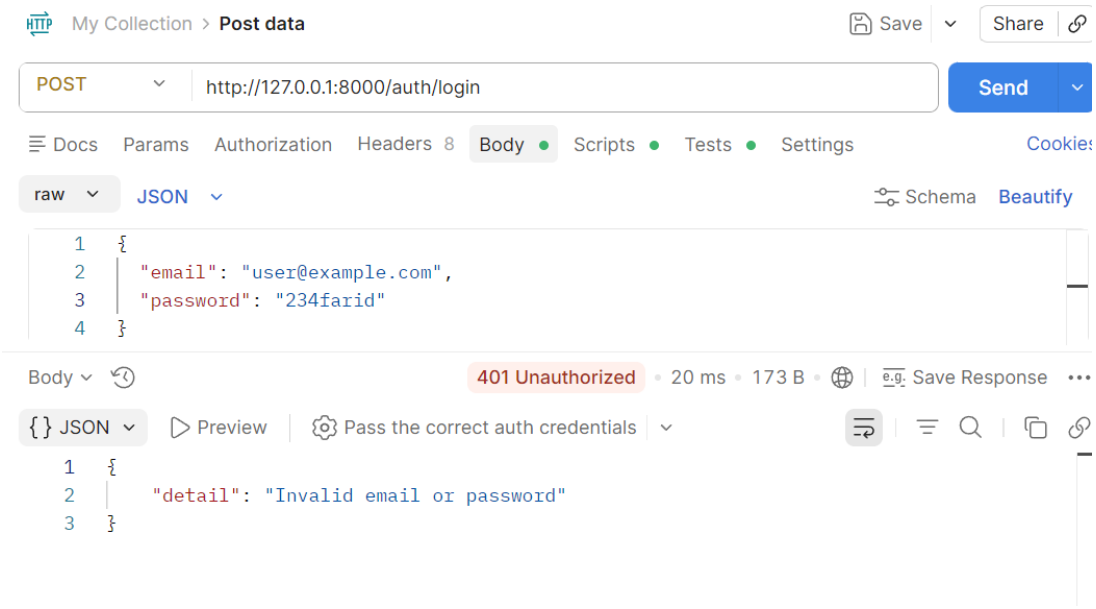


Figure 34: Invalid Login Test

Chapter 4

4. Conclusion and future work

4.1 Conclusion

The Smart Parking System successfully delivers a structured and practical solution for managing modern parking operations. The project combines user authentication, role-based access, parking session handling, zone management, payments, reporting, and image-based plate recognition within a single full-stack application. Through the FastAPI backend, React frontend, and SQLite database, the system provides an organized environment for users, workers, and administrators to perform their tasks efficiently.

The integration of parking workflows with OCR-based plate detection adds practical value to the application and shows how software engineering concepts can be applied to a real-world management problem. Overall, the project demonstrates that the Smart Parking System is functional, scalable, and suitable as a strong foundation for future development and deployment.

4.2 Future Work

To further improve the functionality, security, and overall user experience of the platform, the following enhancements are proposed for future development:

1. **Token-Based Authentication** Replace local session handling with a more secure token-based authentication mechanism.
2. **Improved Plate Detection Accuracy** Enhance the image-processing pipeline by introducing better region detection and optimized OCR preprocessing.
3. **Online Payment Integration** Connect the system with a real payment gateway to support secure and automated parking payments.
4. **Advanced Reporting and Analytics** Add richer dashboards and analytical reports for parking usage, revenue trends, and worker activity.
5. **Production Database Support** Replace SQLite with a more scalable database solution such as PostgreSQL for larger deployments.
6. **Notification and Alert System** Introduce notifications for parking events, overdue payments, violations, and administrative updates.
7. **Deployment and Monitoring Improvements** Extend the project with production-ready deployment, logging, and monitoring tools for long-term maintenance.

Bibliography

1. Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. "You Only Look Once: Unified, Real-Time Object Detection." In: *arXiv* (2015). https://arxiv.org/abs/1506.02640.
2. Ultralytics. (2023). YOLOv8 – Documentation for the YOLOv8 object detection model. Retrieved from https://docs.ultralytics.com/models/yolov8/.
3. Ultralytics. (2023). YOLOv8 Python Package and Pre-trained Models – Official GitHub repository. Retrieved from https://github.com/ultralytics/ultralytics.
4. Satadal Saha. "A Review on Automatic License Plate Recognition System." In: *arXiv* (2019). https://arxiv.org/abs/1902.09385.
5. Rodrigo Laroca, Everson Severo, Luiz A. Zanlorensi, Luiz S. Oliveira, Gabriel R. Gonçalves, William R. Schwartz, and David Menotti. "A Robust Real-Time Automatic License Plate Recognition Based on the YOLO Detector." In: *International Joint Conference on Neural Networks (IJCNN)* (2018). https://ieeexplore.ieee.org/document/8489629.
6. Prapaporn Meesad, Sarayut Intarawiset, and Teerapat Khamket. "Advanced Deep Learning Techniques for Automated License Plate Recognition." In: *Scientific Reports* (2025). https://www.nature.com/articles/s41598-025-24967-9.
7. Jaided AI. (2024). EasyOCR – Ready-to-use OCR with 80+ supported languages. Retrieved from https://github.com/JaidedAI/EasyOCR.
8. OpenCV. (2024). OpenCV – Open Source Computer Vision Library. Retrieved from https://opencv.org/.
9. S. S. Channamallu, A. S. Fahim, and co-authors. "A Review of Smart Parking Systems." In: *Smart Cities* (2023). https://www.sciencedirect.com/science/article/pii/S235214652301219X.
10. Abdulaziz Alharbi, Hany M. Harb, and co-authors. "Web-based Framework for Smart Parking System." In: *International Journal of Information Technology* (2021). https://link.springer.com/article/10.1007/s41870-021-00725-8.
11. D. Mackowski, M. Bai, T. Ouyang, and H. M. Zhang. "Parking Space Management via Dynamic Performance-Based Pricing." In: *arXiv* (2015). https://arxiv.org/abs/1501.00638.
12. FastAPI. (2024). FastAPI – Modern, fast (high-performance) web framework for building APIs with Python. Retrieved from https://fastapi.tiangolo.com/.
13. Pydantic. (2024). Pydantic – Data validation using Python type hints. Retrieved from https://docs.pydantic.dev/.
14. React. (2024). React – The library for web and native user interfaces. Retrieved from https://react.dev/.

15. SQLite. (2024). SQLite – Self-contained, serverless, zero-configuration SQL database engine. Retrieved from https://www.sqlite.org/docs.html.
16. Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. "Array programming with NumPy." In: *Nature* 585 (2020), pp. 357–362. https://doi.org/10.1038/s41586-020-2649-2.
17. Pytest. (2024). Pytest – Helps you write better programs. Retrieved from https://docs.pytest.org/.
18. Vite. (2024). Vite – Next Generation Frontend Tooling. Retrieved from https://vite.dev/.

List of Figures

Acknowledgment

First and foremost, I would like to express my heartfelt gratitude to my supervisor, **Guettala Walid**, for his invaluable guidance throughout the course of this project. His deep expertise in the field, constructive feedback, and constant encouragement have been instrumental in shaping the direction and quality of this thesis. His dedication to mentoring and high standards of academic excellence have continuously inspired me to improve and strive for the best.

I would also like to extend my sincere thanks to my family for their unconditional love, endless patience, and constant support. Their belief in me, especially during moments of challenge and doubt, has been the foundation of my resilience and determination. I am truly grateful for their presence, motivation, and sacrifices, which have empowered me to successfully complete this academic endeavour.

This work is a reflection of the collective support, encouragement, and inspiration I have received from those around me. I am deeply thankful to everyone who played a role in this journey.